APUNTES UNIVERSITARIOS

Foundations of Machine Learning and Deep Learning

Apuntes integrados de teoría y prácticas

20 de octubre de 2026

Índice general

1 Fundamentos del aprendizaje automático y modelos clásicos **2**

1.1 Apuntes del Tema 1: Foundations of Machine Learning — Classical Models & Clustering . . . 2

1.1.1 1. Conceptos Fundamentales y Paradigmas de Aprendizaje . . . 2

1.1.2 2. k-Nearest Neighbors (kNN) . . . 2

1.1.3 3. Naive Bayes . . . 3

1.1.4 4. Árboles de Decisión (Decision Trees) . . . 4

1.1.5 5. Clustering (Agrupamiento No Supervisado) . . . 5

1.1.6 6. Práctica y Aplicación en Código PyTorch / Scikit-Learn . . . 5

1.2 Conexión con las prácticas . . . 8

1.2.1 Fuente práctica: `01_01_warm-up_and_first_model.ipynb` . . . 8

1.2.2 Fuente práctica: `01_02_Naive_Bayes.ipynb` . . . 9

1.2.3 Fuente práctica: `01_03_Decision_trees.ipynb` . . . 10

1.2.4 Fuente práctica: `01_05_Clustering.ipynb` . . . 11

1.3 Autoevaluación razonada . . . 11

1.4 Fuentes y trazabilidad . . . 11

2 Ensembles y descomposición multiclase **13**

2.1 Apuntes del Tema 2: Ensemble Models & Multiclass Decomposition . . . 13

2.1.1 1. Motivación y Métodos de Ensamble (*Ensembles*) . . . 13

2.1.2 2. Bagging y Random Forest . . . 13

2.1.3 3. Boosting: AdaBoost y Gradient Boosting (XGBoost) . . . 14

2.1.4 4. Descomposición Multiclase: OVA y OVO . . . 15

2.1.5 5. Práctica y Aplicación en Código (Scikit-Learn) . . . 15

2.2 Conexión con las prácticas . . . 18

2.2.1 Fuente práctica: `01_04_Ensembles.ipynb` . . . 18

2.3 Autoevaluación razonada . . . 19

2.4 Fuentes y trazabilidad . . . 19

3 Regresión lineal **20**

3.1 Apuntes del Tema 3 y Tema 4: Modelos Lineales — Regresión Lineal y Regresión Logística . . . 20

3.1.1 1. Regresión Lineal . . . 20

3.1.2 2. Regresión Logística . . . 21

3.1.3 3. Escalado de Características y Expansión Polinomial . . . 22

3.1.4 4. Regularización en Modelos Lineales (L_1 y L_2) . . . 22

3.1.5 5. Práctica y Aplicación en Código (PyTorch y Scikit-Learn) . . . 23

3.2 Conexión con las prácticas . . . 25

3.2.1 Fuente práctica: `03_01_Linear_regression.ipynb` . . . 26

3.3 Autoevaluación razonada . . . 26

3.4 Fuentes y trazabilidad . . . 27

4 Regresión logística **28**

4.1 Conexión con las prácticas . . . 28

4.1.1 Fuente práctica: `04_01_Logistic_regression.ipynb` . . . 28

4.2 Autoevaluación razonada . . . 29

4.3 Fuentes y trazabilidad . . . 29

5 Perceptrones multicapa **30**

5.1 Apuntes del Tema 5: Multilayer Perceptron (MLP I y II) . . . 30

5.1.1 1. De Modelos Lineales a No Lineales . . . 30

5.1.2 2. Capas Ocultas y Funciones de Activación . . . 30

5.1.3 3. Clasificación Multiclase: Regresión Softmax . . . 31

5.1.4 4. Algoritmo de Propagación: Forward Pass y Backpropagation . . . 31

5.1.5 5. Propiedades Teóricas e Inicialización de Parámetros . . . 32

5.1.6 6. Problemas en Redes Profundas: Desvanecimiento y Explosión del Gradiente . . . 32

5.1.7 7. Práctica y Aplicación en Código (PyTorch) . . . 33

5.2 Conexión con las prácticas . . . 36

5.2.1 Fuente práctica: `05_MLP_1.ipynb` . . . 36

5.2.2 Fuente práctica: `05_02_MLP.ipynb` . . . 36

5.2.3 Fuente práctica: `05_04_MLP.ipynb` . . . 37

5.2.4 Fuente práctica: `02_01_BasicPytorch.ipynb` . . . 37

5.2.5 Fuente práctica: `02_02_Autograd.ipynb` . . . 37

5.3 Autoevaluación razonada . . . 38

5.4 Fuentes y trazabilidad . . . 38

6 Regularización en redes profundas **39**
6.1 Apuntes del Tema 6: Regularización en Redes Profundas 39
6.1.1 1. El Problema del Sobreajuste (*Overfitting*) en Redes Profundas 39
6.1.2 2. Decaimiento de Pesos (*Weight Decay* / Regularización L_2 vs. L_1) . . 39
6.1.3 3. Dropout (Inyección de Ruido en Activaciones) 40
6.1.4 4. Parada Temprana (*Early Stopping*) 41
6.1.5 5. Práctica y Aplicación en Código (PyTorch) 42
6.2 Conexión con las prácticas 46
6.2.1 Fuente práctica: `06_Regularization_01.ipynb` 46
6.3 Autoevaluación razonada 46
6.4 Fuentes y trazabilidad 47

7 Descenso de gradiente y optimizadores **48**
7.1 Apuntes del Tema 8: Descenso de Gradiente, Métodos Acelerados y Adaptativos 48
7.1.1 1. Fundamentos de Optimización en Deep Learning 48
7.1.2 2. Algoritmos de Descenso de Gradiente 49
7.1.3 3. Optimizadores Acelerados y Adaptativos 50
7.1.4 4. Polítas de Ajuste (*Learning Rate Schedulers*) y Warmup 51
7.1.5 5. Práctica y Aplicación en Código (PyTorch) 52
7.2 Conexión con las prácticas 55
7.2.1 Fuente práctica: `08_01_GradientDescent_Lab.ipynb` 55
7.2.2 Fuente práctica: `08_02_Optimizers_Lab.ipynb` 56
7.2.3 Fuente práctica: `08_03_ExploringTorchOptim.ipynb` 56
7.2.4 Fuente práctica: `02_03_AutogradExercise.ipynb` 56
7.2.5 Fuente práctica: `02_04_Autograd_Lab.ipynb` 57
7.3 Autoevaluación razonada 57
7.4 Fuentes y trazabilidad 57

8 Redes neuronales convolucionales **59**
8.1 Apuntes del Tema 9: Redes Neuronales Convolucionales (CNN I y II) 59
8.1.1 1. De MLPs a Convoluciones: Motivación e Sesgos Inductivos 59
8.1.2 2. Operaciones Convolucionales y sus Parámetros 59
8.1.3 3. Capas de Agregación (*Pooling*) 60
8.1.4 4. Jerarquía de Representaciones y Arquitecturas Clásicas 61
8.1.5 5. Gestión de Datos en PyTorch: `Dataset` y `DataLoader` 61
8.1.6 6. Práctica y Aplicación en Código (PyTorch y Torchvision) 62
8.2 Conexión con las prácticas 65
8.2.1 Fuente práctica: `09_02_CNN.ipynb` 65
8.2.2 Fuente práctica: `09_04_CNN.ipynb` 66

8.2.3 Fuente práctica: `09_06_CNN.ipynb` . . . 66
8.3 Autoevaluación razonada . . . 67
8.4 Fuentes y trazabilidad . . . 67

9 Datos secuenciales y redes recurrentes 68
9.1 Apuntes del Tema 10: Redes Neuronales Recurrentes . . . 68
9.1.1 1. De Datos Estáticos a Datos Secuenciales . . . 68
9.1.2 2. Arquitectura de la Red Neuronal Recurrente Básica (RNN Vanilla) . . . 68
9.1.3 3. Entrenamiento: Retropropagación a través del Tiempo (BPTT) . . . 69
9.1.4 4. Redes Gated: LSTM y GRU . . . 70
9.1.5 5. RNNs Bidireccionales y Apiladas (*Deep RNNs*) . . . 71
9.1.6 6. Práctica y Aplicación en Código (PyTorch) . . . 72
9.2 Conexión con las prácticas . . . 75
9.2.1 Fuente práctica: `10_01_SequentialData.ipynb` . . . 75
9.2.2 Fuente práctica: `10_02_RNNfromScratch.ipynb` . . . 76
9.3 Autoevaluación razonada . . . 76
9.4 Fuentes y trazabilidad . . . 76

10 Atención y Transformers 77
10.1 Apuntes del Tema 11: Attention Mechanisms and Transformers . . . 77
10.1.1 1. Introducción y Limitaciones de los Modelos Secuenciales (Seq2Seq) . . . 77
10.1.2 2. Mecanismos de Atención en RNNs (Bahdanau et al., 2014) . . . 77
10.1.3 3. Autoatención (*Self-Attention*) en Transformers . . . 78
10.1.4 4. Atención Multicabeza (*Multi-Head Self-Attention*) . . . 79
10.1.5 5. Arquitectura del Transformer: Encoder . . . 80
10.1.6 6. Arquitectura del Transformer: Decoder . . . 81
10.1.7 7. Consejos de Entrenamiento, Inferencia y Arquitecturas . . . 81
10.1.8 8. Práctica en Código (PyTorch) . . . 82
10.2 Conexión con las prácticas . . . 84
10.2.1 Fuente práctica: `11_01_Attention.ipynb` . . . 84
10.2.2 Fuente práctica: `11_02_Attention.ipynb` . . . 85
10.2.3 Fuente práctica: `11_03_Attention.ipynb` . . . 86
10.2.4 Fuente práctica: `11_04_Submission_task.ipynb` . . . 87
10.3 Autoevaluación razonada . . . 88
10.4 Fuentes y trazabilidad . . . 88

11 Sistemas de recomendación 89
11.1 Conexión con las prácticas . . . 89
11.1.1 Fuente práctica: `13_01_Recommender_systems.ipynb` . . . 89
11.2 Autoevaluación razonada . . . 90
11.3 Fuentes y trazabilidad . . . 90

1 Fundamentos del aprendizaje automático y modelos clásicos

1.1 Apuntes del Tema 1: Foundations of Machine Learning — Classical Models & Clustering

1.1.1 1. Conceptos Fundamentales y Paradigmas de Aprendizaje

1.1.1.1 1.1 Trazabilidad y Definición

- **Machine Learning:** Disciplina enfocada en la creación de sistemas capaces de realizar tareas mediante el reconocimiento de patrones y la toma de decisiones basadas en datos de experiencia (ejemplos), evaluando el desempeño mediante la capacidad de **generalización** hacia nuevos datos no vistos y previniendo el *overfitting*.
- **Estructura de Datos Básica:**
- **Dataset:** Conjunto de observaciones $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{n}$.
- **Instancia (x):** Vector de características de dimensión d, donde $x \in \mathbb{R}^d$.
- **Etiqueta/Target (y):** Valor objetivo o clase a predecir.
- **Modelo Paramétrico:** Función de mapeo $f(x;\theta) \to \hat{y}$ parametrizada por pesos y sesgos $\theta = (w, b)$.

1.1.1.2 1.2 Paradigmas de Aprendizaje

- **Supervisado:** Aprendizaje mediante pares entrada-salida etiquetados (x, y). Se divide en:
- **Regresión:** El target y es un valor continuo.
- **Clasificación:** El target y representa una categoría discreta.
- **No Supervisado:** Descubrimiento de estructuras implícitas o agrupamientos en datos sin etiquetas x (ej. *Clustering*, reducción de dimensionalidad).
- **Otros Paradigmas Relevantes:** *Semi-supervised*, *Self-supervised* (pretext tasks), *Transfer Learning* (reutilización de backbones preentrenados), *Active Learning*, *Online/Incremental Learning* y *Federated Learning*.

1.1.2 2. k-Nearest Neighbors (kNN)

1.1.2.1 2.1 Principio e Intuición

- **Algoritmo Lazy Learner:** No requiere una fase de entrenamiento o ajuste de parámetros explícito ($f(x)$ se evalúa en tiempo de consulta).
- **Regla de Clasificación:** Asigna la etiqueta a una muestra de prueba x_{query} según el voto mayoritario (o promedio ponderado por la distancia inversa) de sus k vecinos más cercanos en el espacio métrico.

1.1.2.2 2.2 Métricas de Distancia

- **Distancia Euclídea (L_2):**

$$\{}text{dist}_{\{}text{Euclidean}}(x, x') = \{}sqrt{\{}sum_{j=1}^{d} (x_j - x_j')^2}$$

- **Distancia Manhattan (L_1):**

$$\{}text{dist}_{\{}text{Manhattan}}(x, x') = \{}sum_{j=1}^{d} \{}vert{}x_j - x_j'\{}vert{}$$

- **Minkowski (L_p):**

$$\{}text{dist}_{L^p}(x, x') = \{}left(\{}sum_{j=1}^{d} \{}vert{}x_j - x_j'\{}vert{}^p \{}right)^{1/p}$$

1.1.2.3 2.3 Selección de k y Efectos de Escala

- **Efecto de k:**
- $k = 1$ (Bajo): Frontera extremadamente flexible, muy sensible al ruido y propenso a sobreajuste (*overfitting*).
- k elevado: Fronteras muy suaves, riesgo de subajuste (*underfitting*) al mezclar vecindarios.
- **Necesidad de Normalización:** Dado que kNN depende de distancias geométricas, las variables con rangos numéricos elevados dominan el cálculo. Es indispensable estandarizar las características ($x' = \frac{x-\mu}{\sigma}$) antes de calcular las distancias.
- **La Maldición de la Dimensionalidad (*Curse of Dimensionality*):** A medida que la dimensión d del espacio crece, el volumen aumenta exponencialmente, volviendo los datos muy dispersos y reduciendo la capacidad discriminativa de las distancias geométricas entre los puntos.

1.1.3 3. Naive Bayes

1.1.3.1 3.1 Fundamento Probabilístico Generativo

- **Modelo Generativo:** Modela la probabilidad de la clase a priori $P(y)$ y la verosimilitud de los datos $P(x|y)$ para estimar la probabilidad a posteriori $P(y|x)$ aplicando el Teorema de Bayes:

$$P(y\{}vert{}x) = \{}frac{P(x\{}vert{}y)P(y)}{P(x)} \{}propto P(y) \{}prod_{j=1}^{d} P(x_j\{}vert{}y)$$

- **Regla MAP (Maximum A Posteriori):** Selección de la clase que maximiza el producto:

$$\{}hat{y} = \{}arg\{}max_{y} P(y) \{}prod_{j=1}^{d} P(x_j\{}vert{}y)$$

- **Asunción "Naive" (Ingenua):** Presupone la **independencia condicional** de todas las características x_j dada la clase y.

1.1.3.2 3.2 Variantes Principales de Naive Bayes

| Variante | Tipo de Variable (x_j) | Estimación de Verosimilitud $P(x_j|y)$ |

| :— | :— | :— |

| **Categorical NB** | Categóricas/Nominales discretas | $P(x_j = v \mid y) = \frac{\text{count}(x_j=v,y)+\alpha}{\text{count}(y)+\alpha \cdot K_j}$ |

| **Multinomial NB** | Conteos discretos (frecuencias de texto/Bag-of-Words) | $P(x_j \mid y) = \frac{\text{count}(x_j \text{ en clase } y)+\alpha}{\sum_k \text{count}(x_k \text{ en clase } y)+\alpha \cdot V}$ |

| **Gaussian NB** | Características continuas | $P(x_j \mid y) = \frac{1}{\sqrt{2\pi\sigma_{j,y}^2}} \exp\left(-\frac{(x_j-\mu_{j,y})^2}{2\sigma_{j,y}^2}\right)$ |

- **Laplace Smoothing ($\alpha = 1$):** Corrige el problema de la **probabilidad cero** (frecuencia nula en datos de entrenamiento que anularía todo el producto de verosimilitudes) sumando el parámetro α en el numerador y $\alpha \cdot K_j$ (o $\alpha \cdot V$) en el denominador.

1.1.4 4. Árboles de Decisión (Decision Trees)

1.1.4.1 4.1 Concepto y Particionado Recursivo

- Modelos no paramétricos e interpretables para clasificación y regresión que dividen recursivamente el espacio de características mediante reglas ortogonales a los ejes.

1.1.4.2 4.2 Comparativa de Algoritmos y Criterios de División

1. **ID3 (Iterative Dichotomiser 3):**

- Solo admite atributos categóricos y no realiza poda.
- Utiliza la **Entropía** $H(S)$ y la **Ganancia de Información** $IG(S, A)$:

$$H(S) = - \{}sum_{i=1}^{c} p_i \{}log_2(p_i)$$

$$IG(S, A) = H(S) - \{}sum_{v \{}in \{}text{Valores}(A)} \{}frac{\{}vert{}S_v\{}vert{}}{\{}vert{}S\{}v
H(S_v)$$

1. **C4.5:**

- Soporta variables continuas mediante la evaluación de umbrales binarios de corte t.
- Sustituye el IG por el **Gain Ratio** para corregir el sesgo de ID3 hacia atributos con excesivos valores únicos (como un `ID_Cliente`):

$$\{}text{GainRatio}(S, A) = \{}frac{IG(S, A)}{\{}text{SplitInfo}(A)}$$

$$\{}text{SplitInfo}(A) = -\{}sum_{i=1}^{k} \{}frac{\{}vert{}S_i\{}vert{}}{\{}vert{}S\{}vert{}}
\{}log_2\{}left(\{}frac{\{}vert{}S_i\{}vert{}}{\{}vert{}S\{}vert{}}\{}right)$$

- Aplica poda pesimista posterior al entrenamiento.

1. **CART (Classification and Regression Trees):**

- Realiza exclusivamente divisiones binarias.
- Minimiza la **Impureza de Gini** en clasificación:

$$G(S) = 1 - \{}sum_{i=1}^{c} p_i^2$$

- En regresión, minimiza la varianza/suma de errores cuadráticos (SSE).

1.1.5 5. Clustering (Agrupamiento No Supervisado)

1.1.5.1 5.1 k-means

- **Objetivo:** Particionar X en c clusters disjuntos $\omega_1, \ldots, \omega_c$ minimizando la varianza dentro de cada cluster (suma de distancias al cuadrado al centroide μ_i).
- **Paso a Paso del Algoritmo:**

1. Inicialización aleatoria de k centroides.
2. Asignación de cada punto al centroide más cercano.
3. Reordenamiento y recalculo del punto medio/centroide de cada cluster.
4. Repetición iterativa hasta convergencia.

- **Selección de k:** Se evalúa empíricamente mediante el **Elbow Method** (buscando el punto de inflexión en la curva de la inercia/varianza intra-cluster).

1.1.5.2 5.2 Clustering Jerárquico Aglomerativo (Agglomerative)

- **Mecanismo:** Construye una estructura en árbol (*Dendrograma*) fusionando sucesivamente los dos clusters más cercanos desde un estado inicial donde cada punto es un cluster individual.
- **Criterios de Enlace (Linkage Criteria):**
- **Single Linkage:** Distancia mínima entre miembros de dos clusters ($d_{\min}(C_1, C_2)$).
- **Complete Linkage:** Distancia máxima entre miembros de dos clusters ($d_{\max}(C_1, C_2)$).
- **Average Linkage:** Distancia promedio entre todos los pares de elementos entre clusters.

1.1.6 6. Práctica y Aplicación en Código PyTorch / Scikit-Learn

```
1
2  import numpy as np
3
4  import torch
5
6  import torch.nn as nn
7
8  import torch.optim as optim
9
10 from sklearn.neighbors import KNeighborsClassifier
11
```

```python
from sklearn.naive_bayes import GaussianNB

from sklearn.tree import DecisionTreeClassifier

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler



# ------------------------------------------------------------------------

# 1. Modelos Clásicos en Scikit-Learn (kNN, Naive Bayes, Decision Trees)

# ------------------------------------------------------------------------

def train_classical_models(X_train, y_train, X_test):

    # kNN: requiere estandarización previa de características

    scaler = StandardScaler()

    X_train_scaled = scaler.fit_transform(X_train)

    X_test_scaled = scaler.transform(X_test)



    knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')

    knn.fit(X_train_scaled, y_train)



    # Gaussian Naive Bayes

    gnb = GaussianNB()

    gnb.fit(X_train, y_train)



    # Árbol de Decisión CART (Criterio Gini)

    tree = DecisionTreeClassifier(criterion='gini', max_depth=5)

    tree.fit(X_train, y_train)



    return knn, gnb, tree



# ------------------------------------------------------------------------

# 2. Clustering (k-means)

```

```python
# ------------------------------------------------------------------------

def run_kmeans_clustering(X_data, n_clusters=3):

    kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)

    cluster_labels = kmeans.fit_predict(X_data)

    centroids = kmeans.cluster_centers_

    return cluster_labels, centroids



# ------------------------------------------------------------------------

# 3. MLP Baseline con Vectorización (Aplanado) en PyTorch

# ------------------------------------------------------------------------

class MLPClassifier(nn.Module):

    """

    MLP Baseline para imágenes o datos vectorizados.

    Demuestra la pérdida de estructura espacial al aplanar un tensor (C, H, W) -> (C * H * W)

    """

    def __init__(self, input_shape=(3, 64, 64), num_classes=10, hidden_units=[512, 256], dropout_rate=0.3):

        super(MLPClassifier, self).__init__()



        # Dimensión de la entrada aplanada

        self.input_dim = input_shape[0] * input_shape[1] * input_shape[2]



        layers = []

        in_dim = self.input_dim



        for h_dim in hidden_units:

            layers.append(nn.Linear(in_dim, h_dim))

            layers.append(nn.ReLU())

            layers.append(nn.Dropout(p=dropout_rate))

```

```
            in_dim = h_dim


        # Capa lineal de salida (retorna logits para CrossEntropyLoss)
        layers.append(nn.Linear(in_dim, num_classes))


        self.network = nn.Sequential(*layers)


    def forward(self, x):
        # Aplanado explícito: (Batch, Channels, Height, Width) -> (Batch, C * H * W)
        x = x.view(x.size(0), -1)
        return self.network(x)


if __name__ == "__main__":
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    dummy_images = torch.randn(16, 3, 64, 64).to(device)
    model = MLPClassifier(input_shape=(3, 64, 64), num_classes=10).to(device)
    logits = model(dummy_images)
    print(f"Forma de la entrada : {dummy_images.shape}")
    print(f"Forma de los logits : {logits.shape}")
```

1.2 Conexión con las prácticas

Las prácticas de Practice_01 llevan los conceptos de representación, clasificación y clustering a datos ejecutables. Los gráficos y salidas se describen únicamente cuando están almacenados en los notebooks.

1.2.1 Fuente práctica: `01_01_warm-up_and_first_model.ipynb`

Ruta: code/Practice_01/01_01_warm-up_and_first_model.ipynb

Apartados detectados: 01_01 Warm-up and first model; The basics of NumPy; 1) Creating arrays and dtypes; 2) Shapes, dimensions, and reshape; 3) Indexing, slicing, and boolean masks.

Salidas gráficas almacenadas: 10.

Salida textual conservada:

a: [1 2 3] dtype: int64 shape: (3,) | b: [1. 2. 3.] dtype: float64 shape: (3,) | zeros: | [[0. 0. 0.]

Fragmento representativo:

```

import numpy as np

# 1) From lists
a = np.array([1, 2, 3])                 # default dtype = int
b = np.array([1.0, 2.0, 3.0])           # dtype = float

# 2) Constructors
z = np.zeros((2, 3))                    # 2x3 zeros, float64 by default
o = np.ones((3, 2), dtype=np.int32)     # 3x2 ones, explicitly int32
ar = np.arange(0, 10, 2)                # 0, 2, 4, 6, 8
ls = np.linspace(0, 1, 5)               # 5 evenly spaced values between 0 and 1

# 3) Random arrays
u = np.random.rand(2, 4)                # uniform [0,1), shape (2,4)
n = np.random.randn(3, 3)               # standard normal, shape (3,3)

print("a:", a, "dtype:", a.dtype, "shape:", a.shape)
print("b:", b, "dtype:", b.dtype, "shape:", b.shape)
print("zeros:\n", z)
print("ones:\n", o)
print("arange:", ar)
print("linspace:", ls)
print("uniform rand:\n", u)
print("normal rand:\n", n)
```

1.2.2 Fuente práctica: 01_02_Naive_Bayes.ipynb

Ruta: code/Practice_01/01_02_Naive_Bayes.ipynb

Apartados detectados: 01_02 Naive Bayes; Gaussian Naive Bayes on Wine dataset; Step 1: Load the Wine dataset and keep NumPy arrays; Step 2: Train a Gaussian Naive Bayes model; Step 3: Use ‘predict_proba‘ to see class probabilities.

Salidas gráficas almacenadas: 2.

Salida textual conservada:

[0 0 0

```
8
9  X = df.iloc[:,:-1]
10 #print(X.head())
11 y = df['target']
12 #print(y.head)
13 y_names = wine.target_names
14 #print(y_names)
```

1.2.3 Fuente práctica: 01_03_Decision_trees.ipynb

Ruta: code/Practice_01/01_03_Decision_trees.ipynb

Apartados detectados: 01_03 Decision trees; 1. Decision trees; Step 1: Train a baseline Decision Tree on Breast Cancer; Step 2: Depth vs. performance; Step 3: Visualize the learned tree (structure and rules).

Salidas gráficas almacenadas: 5.

Salida textual conservada:

> mean radius mean texture mean perimeter mean area mean smoothness \{} | 0 17.99 10.38 122.80 1001.0 0.11840 | 1 20.57 17.77 132.90 1326.0 0.08474 | 2 19.69 21.25 130.00 1203.0 0.10960

Fragmento representativo:

```
1
2  #1. Load the dataset with [`load_breast_cancer()`](https://scikit-learn.org/stable/
       modules/generated/sklearn.datasets.load_breast_cancer.html).
3  import pandas as pd
4  from sklearn.datasets import load_breast_cancer
5  data = load_breast_cancer()
6
7  display(pd.DataFrame(data.data, columns=data.feature_names).head())  # Display the
       dataset description
8  #2. Split into train/test (70/30) with `train_test_split(..., random_state=42,
       stratify=y)`.
9  from sklearn.model_selection import train_test_split
10 X_train, X_test, y_train, y_test = train_test_split(data.data, data.target, test_size
       =0.3, random_state=42, stratify=data.target)
11
12 #3. Create `DecisionTreeClassifier(random_state=42)` and train it on the training set.

13 from sklearn.tree import DecisionTreeClassifier
14 tree = DecisionTreeClassifier(random_state=42)
15 tree.fit(X_train, y_train)
16 #4. Print training and test accuracies.
17 train_accuracy = tree.score(X_train, y_train)
18 test_accuracy = tree.score(X_test, y_test)
19 print(f"Training accuracy: {train_accuracy:.2%}")
20 print(f"Test accuracy: {test_accuracy:.2%}")
21 #5. Keep `feature_names` and `class_names` handy (’well use them for visualization).
22 feature_names = data.feature_names
23 class_names = data.target_names
```

1.2.4 Fuente práctica: 01_05_Clustering.ipynb

Ruta: code/Practice_01/01_05_Clustering.ipynb

Apartados detectados: 01_04 Clustering; 1. K-means; 1.1 Load a custom dataset; 1.2 K-Means randomness: different runs, different results; 1.3 Multiple runs with ‘n_init‘.

Salidas gráficas almacenadas: 14.

Salida textual conservada:

> <Figure size 640x480 with 1 Axes> | <Figure size 2000x800 with 10 Axes> | <Figure size 800x600 with 1 Axes> | Single-run inertia: 67.51

Fragmento representativo:

```

from sklearn.datasets import make_blobs

import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=[100, 50, 5], centers=[(0,0), (3,3), (-2,-2)],
    cluster_std=[0.4, 0.2, 0.1], random_state=42)


#3. Plot a scatter of the points (no labels).
plt.scatter(X[:, 0], X[:, 1], edgecolor="k")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

1.3 Autoevaluación razonada

Pregunta

¿Por qué se separan entrenamiento, validación y prueba?

Solución. Para estimar la generalización sin usar la prueba durante el ajuste ni durante la selección de hiperparámetros.

Pregunta

¿Qué diferencia esencial hay entre clasificación y clustering?

Solución. La clasificación usa etiquetas objetivo; el clustering busca estructura en datos sin etiquetas.

1.4 Fuentes y trazabilidad

Diapositivas:

- slides/01_01_Introduction1.pdf
- slides/01_02_Introduction2.pdf

- `slides/01\_03\_Introduction3.pdf`
- `slides/01\_04\_Introduction4.pdf`
- `slides/01\_05\_Introduction5.pdf`
- `slides/02\_MathematicalFoundations.pdf`

Nota personal: apuntes/Apuntes del Tema 1 - Foundations of Machine Learning — Classical Models & Clustering.md

2 Ensembles y descomposición multiclase

2.1 Apuntes del Tema 2: Ensemble Models & Multiclass Decomposition

2.1.1 1. Motivación y Métodos de Ensamble (*Ensembles*)

2.1.1.1 1.1 Filosofía del Ensamble

- **Limitaciones de un Modelo Individual:**
- Un modelo demasiado simple puede subajustar (*underfit*), mostrando un alto sesgo (*bias*) y bajo rendimiento.
- Un modelo excesivamente complejo puede sobreajustar (*overfit*), memorizando el ruido de los datos de entrenamiento y mostrando alta varianza (*variance*).
- Algunos modelos (como los árboles de decisión no podados) son inestables: pequeñas variaciones en los datos de entrenamiento generan cambios drásticos en la estructura del árbol.
- **Idea Clave:** Combinar múltiples estimadores débiles (*weak learners*) para construir un estimador fuerte (*strong learner*) cuya predicción colectiva sea más precisa, estable y robusta que cualquiera de los modelos individuales por separado.
- **Diversidad:** Para que un ensamble funcione eficazmente, los estimadores individuales deben cometer errores de manera independiente o desacoplada (diversidad de hipótesis).

2.1.2 2. Bagging y Random Forest

2.1.2.1 2.1 Bagging (*Bootstrap Aggregating*)

- **Objetivo:** Reducir la **varianza** de modelos inestables y con alta varianza (ej. árboles de decisión profundos) sin incrementar su sesgo.
- **Mecanismo Paso a Paso:**

1. **Muestreo Bootstrap:** Dado un dataset $\mathcal{D}$ de N muestras, se generan B subconjuntos $\mathcal{D}_b$ extrayendo N muestras con reemplazo de $\mathcal{D}$.
2. **Entrenamiento Independiente:** Se entrena un estimador $h_b(x)$ de forma completamente independiente sobre cada subconjunto $\mathcal{D}_b$.

3. **Agregación:**

- **Clasificación:** Voto mayoritario:

$$\{}hat{y} = \{}arg\{}max_{c} \{}sum_{b=1}^{B} \{}mathbb{I}(h_b(x) = c)$$

- **Regresión:** Promedio simple:

$$\{}hat{y} = \{}frac{1}{B} \{}sum_{b=1}^{B} h_b(x)$$

2.1.2.2 2.2 Random Forest

- **Mejora sobre Bagging:** En el Bagging estándar, si existe una característica dominante, todos los árboles la elegirán en la raíz, resultando en un conjunto de árboles altamente correlacionados.
- **Decorrelación de Árboles:** Random Forest añade una segunda fuente de aleatoriedad: en cada división (*split*) de cada árbol, se selecciona al azar un subconjunto reducido de m características de las d totales disponibles ($m \ll d$).
- Para clasificación: típicamente $m = \sqrt{d}$.
- Para regresión: típicamente $m = \frac{d}{3}$.
- **Evaluación Out-Of-Bag (OOB):** Dado que cada muestra *bootstrap* deja fuera aproximadamente un 36.8% de los datos ($1 - 1/e \approx 0.632$ incluidos), estas muestras "no vistas" sirven como conjunto de validación implícito (*OOB score*) sin requerir un split de validación adicional.

2.1.3 3. Boosting: AdaBoost y Gradient Boosting (XGBoost)

2.1.3.1 3.1 AdaBoost (*Adaptive Boosting*)

- **Mecanismo:** Entrena estimadores de forma **secuencial**. Mantiene una distribución de pesos $D_t(i)$ sobre las muestras del dataset.
- **Algoritmo Paso a Paso:**

1. Inicializa los pesos de las muestras uniformemente: $D_1(i) = 1/N$.
2. Para cada iteración $t = 1, \ldots, T$:

- Entrena un estimador débil $h_t(x)$ minimizando el error ponderado:

$$\{}epsilon_t = \{}sum_{i: y_i \{}neq h_t(x_i)} D_t(i)$$

- Calcula la importancia/confianza del estimador en la decisión final:

$$\{}alpha_t = \{}frac{1}{2} \{}ln\{}left(\{}frac{1 - \{}epsilon_t}{\{}epsilon_t}\{}right)$$

- Actualiza los pesos de las muestras incrementando la importancia de las mal clasificadas:

$$D_{t+1}(i) \{}propto D_t(i) \{}cdot \{}exp\{}left(-\{}alpha_t y_i h_t(x_i)\{}right)$$

- Renormaliza D_{t+1} para que forme una distribución válida que sume 1.

1. **Predicción Final:** Voto ponderado:

$$\{}hat{y} = \{}text{signo}\{}left(\{}sum_{t=1}^{T} \{}alpha_t h_t(x)\{}right)$$

2.1.3.2 3.2 Gradient Boosting y XGBoost

- **Concepto Clave:** En lugar de reponderar muestras, cada nuevo árbol $h_t(x)$ se ajusta directamente para predecir los **residuos** (o el gradiente negativo de la función de pérdida) del ensamble acumulado $F_{t-1}(x)$.
- **Ajuste Aditivo:**

$$F_t(x) = F_{t-1}(x) + \{}nu h_t(x)$$

donde $\nu \in (0, 1]$ es el parámetro de tasa de aprendizaje (*shrinkage*) que regula la contribución de cada nuevo árbol para evitar el sobreajuste.

- **Residuos para Error Cuadrático Medio (MSE):**

$$r_{i, t-1} = -\{}left[\{}frac{\{}partial \{}mathcal{L}(y_i, F(x_i))}{\{}partial F(x_i)} \{}right]_{F = F_{t-1}} = y_i - F_{t-1}(x_i)$$

- **XGBoost (*Extreme Gradient Boosting*):** Extensión altamente eficiente que incluye:
- Expansión de Taylor de segundo orden (utiliza tanto el gradiente de primer orden g_i como el Hessiano h_i).
- Regularización L_1 (α) y L_2 (λ) sobre las hojas del árbol.
- Manejo automático e intrínseco de valores faltantes (*sparse-aware split finding*).

2.1.4 4. Descomposición Multiclase: OVA y OVO

Cuando un modelo de clasificación binario puro (ej. Regresión Logística, SVM) debe aplicarse a un problema de K clases, se emplean estrategias de descomposición:

| Estrategia | Cantidad de Clasificadores | Configuración del Entrenamiento | Mecanismo de Predicción |

| :— | :— | :— | :— |

| **One-vs-All (OVA)** *(One-vs-Rest)* | K clasificadores | El clasificador k se entrena con la clase k como positiva (+1) y todas las demás como negativas (-1). | Se asigna la clase cuyo clasificador entregue la mayor probabilidad o puntuación (*logit*): $\hat{y} = \arg\max_k f_k(x)$. |

| **One-vs-One (OVO)** | $\frac{K(K-1)}{2}$ clasificadores | Se entrena un clasificador para cada par único de clases (c_i, c_j) ignorando el resto. | Cada sub-clasificador emite un voto por su clase preferida. Gana la clase con **mayor número de votos acumulados**. |

2.1.5 5. Práctica y Aplicación en Código (Scikit-Learn)

```

import numpy as np

from sklearn.datasets import make_classification

```

```
from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier,
    AdaBoostClassifier

from sklearn.multiclass import OneVsRestClassifier, OneVsOneClassifier

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score



# ------------------------------------------------------------------------

# 1. Generación de Dataset Sintético Multiclase (K = 4 clases)

# ------------------------------------------------------------------------

X, y = make_classification(

    n_samples=1000,

    n_features=20,

    n_informative=12,

    n_classes=4,

    random_state=42

)



X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42, stratify=y

)



# ------------------------------------------------------------------------

# 2. Comparativa de Ensembles (Random Forest, AdaBoost, Gradient Boosting)

# ------------------------------------------------------------------------

def train_and_evaluate_ensembles(X_tr, y_tr, X_te, y_te):

    models = {

        "Random Forest": RandomForestClassifier(

            n_estimators=100, max_features="sqrt", random_state=42

        ),
```

```
        "AdaBoost": AdaBoostClassifier(
            n_estimators=50, learning_rate=0.1, random_state=42
        ),
        "Gradient Boosting": GradientBoostingClassifier(
            n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42
        )
    }

    print("--- Evaluación de Métodos de Ensamble ---")
    for name, model in models.items():
        model.fit(X_tr, y_tr)
        preds = model.predict(X_te)
        acc = accuracy_score(y_te, preds)
        print(f"Modelo: {name:<20} | Test Accuracy: {acc:.4f}")


# ------------------------------------------------------------------------
# 3. Estrategias de Descomposición Multiclase (OVA vs OVO)
# ------------------------------------------------------------------------
def train_multiclass_decomposition(X_tr, y_tr, X_te, y_te):
    # Clasificador base binario
    base_binary_clf = SVC(kernel='linear', C=1.0, random_state=42)

    # 1. One-vs-All (OVA / OvR)
    ova_model = OneVsRestClassifier(base_binary_clf)
    ova_model.fit(X_tr, y_tr)
    ova_preds = ova_model.predict(X_te)

    # 2. One-vs-One (OVO)
    ovo_model = OneVsOneClassifier(base_binary_clf)
```

```
121
122     ovo_model.fit(X_tr, y_tr)
123
124     ovo_preds = ovo_model.predict(X_te)
125
126
127
128     print("\n--- Estrategias de Descomposición Multiclase ---")
129
130     print(f"Número de estimadores OVA (K=4)    : {len(ova_model.estimators_)}")
131
132     print(f"Accuracy OVA                       : {accuracy_score(y_te, ova_preds):.4f}
        ")
133
134
135
136     print(f"Número de estimadores OVO (K*(K-1)/2): {len(ovo_model.estimators_)}")
137
138     print(f"Accuracy OVO                       : {accuracy_score(y_te, ovo_preds):.4f}
        ")
139
140
141
142 if __name__ == "__main__":
143
144     train_and_evaluate_ensembles(X_train, y_train, X_test, y_test)
145
146     train_multiclass_decomposition(X_train, y_train, X_test, y_test)
```

2.2 Conexión con las prácticas

El notebook de ensembles conecta la combinación de estimadores con la evaluación empírica y permite contrastar estrategias OVA y OVO cuando aparecen varias clases.

2.2.1 Fuente práctica: `01_04_Ensembles.ipynb`

Ruta: code/Practice_01/01_04_Ensembles.ipynb

Apartados detectados: 01_04 Ensembles; 1. Bagging and Random Forest; Step 1.1: Bagging with Decision Trees; Step 1.2: Random Forest (bagging + random subspace); Step 1.3: Study Random Forest parameters.

Salidas gráficas almacenadas: 7.

Salida textual conservada:

> Training Accuracy: 1.0000 | Testing Accuracy: 0.9591 | Random Forest Training Accuracy: 1.0000 | Random Forest Testing Accuracy: 0.9357

Fragmento representativo:

```
1
2 import pandas as pd
3 from sklearn.tree import DecisionTreeClassifier
4 from sklearn.datasets import load_breast_cancer
```

```python
from sklearn.ensemble import BaggingClassifier
from sklearn.model_selection import train_test_split

# Create a decision tree classifier
dt = DecisionTreeClassifier(random_state=42)

# Create a bagging classifier
bg = BaggingClassifier(
    estimator=dt,
    random_state=42
)

# Load the breast cancer dataset
data = load_breast_cancer()

X_train, X_test, y_train, y_test = train_test_split(
    data.data,
    data.target,
    test_size=0.3,
    random_state=42,
    stratify=data.target
)

# Fit the bagging classifier
bg.fit(X_train, y_train)
```

2.3 Autoevaluación razonada

Pregunta

¿Qué aporta bagging?

Solución. Reduce la varianza al entrenar modelos sobre muestras bootstrap y combinar sus predicciones.

Pregunta

¿Cuántos clasificadores binarios requiere OVO para K clases?

Solución. $K(K-1)/2$, uno por cada pareja de clases.

2.4 Fuentes y trazabilidad

Nota personal: apuntes/Apuntes del Tema 2 - Ensemble Models & Multiclass Decomposition.md

3 Regresión lineal

3.1 Apuntes del Tema 3 y Tema 4: Modelos Lineales — Regresión Lineal y Regresión Logística

3.1.1 1. Regresión Lineal

3.1.1.1 1.1 Introducción y Planteamiento del Modelo

- **Definición:** Modelo paramétrico supervisado para predecir un valor objetivo continuo $y \in \mathbb{R}$ a partir de un vector de características $x \in \mathbb{R}^d$.
- **Ecuación del Modelo:**

$$\hat{y} = w_1 x_1 + w_2 x_2 + \dots + w_d x_d + b = x^\top w + b$$

donde $w \in \mathbb{R}^d$ es el vector de pesos (pesos de las características) y $b \in \mathbb{R}$ es el sesgo (*bias* o intersección).

- **Interpretación Geométrica:** Representa el ajuste de un hiperplano d-dimensional embebido en un espacio de dimensión $\mathbb{R}^{d+1}$:
- $d = 1$: Una recta de regresión.
- $d = 2$: Un plano de regresión.

3.1.1.2 1.2 Función de Pérdida (Error Cuadrático Medio - MSE)

Para medir la discrepancia entre las predicciones $\hat{y}^{(i)}$ y los valores reales $y^{(i)}$, se define el residuo $r^{(i)} = y^{(i)} - \hat{y}^{(i)}$.

La función de pérdida promedio sobre un dataset de n muestras es el **MSE (Mean Squared Error)**:

$$\mathcal{L}(w, b) = \frac{1}{n} \sum_{i=1}^{n} \left(y^{(i)} - \hat{y}^{(i)}\right)^2 = \frac{1}{n} \Vert y - (Xw + b\mathbf{1}) \Vert_2^2$$

3.1.1.3 1.3 Visión Probabilística: Derivación desde Máxima Verosimilitud (MLE)

El uso del error cuadrático medio no es arbitrario; surge de asumir que el valor real y se genera mediante una relación lineal determinista más un **ruido gaussiano** centrado en cero $\epsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$:

$$y^{(i)} = (x^{(i)})^\top w + b + \epsilon^{(i)} \implies y^{(i)} \mid x^{(i)} \sim \mathcal{N}\left((x^{(i)})^\top w + b, \sigma^2\right)$$

Al maximizar la función de Log-Verosimilitud (*Log-Likelihood*) de este modelo bajo muestras independientes e idénticamente distribuidas (*i.i.d.*), la maximización equivale exactamente a la **minimización del MSE**.

3.1.1.4 1.4 Solución Analítica vs. Descenso de Gradiente

- **Solución Analítica (Ecuación Normal):**

Añadiendo una columna de unos a la matriz de características ($\tilde{X} = [\mathbf{1}, X]$), se obtiene el mínimo global en un solo paso:

$$\{}tilde{w} = (\{}tilde{X}^\{}top \{}tilde{X})^{-1} \{}tilde{X}^\{}top y$$

- *Inconveniente:* Invertir la matriz $\tilde{X}^\top \tilde{X}$ requiere un coste computacional $O(d^3)$, siendo inviable para dimensiones elevadas (d muy grande).
- **Descenso de Gradiente (Iterativo):**

Actualización iterativa mediante las derivadas parciales de la pérdida:

$$\{}frac{\{}partial \{}mathcal{L}}{\{}partial w_j} = -\{}frac{2}{n} \{}sum_{i=1}^{n} \{}left(y^{(i)} - \{}hat{y}^{(i)}\{}right) x_j^{(i)}, \{}quad \{}frac{\{}partial \{}mathcal{L}}{\{}partial b} = -\{}frac{2}{n} \{}sum_{i=1}^{n} \{}left(y^{(i)} - \{}hat{y}^{(i)}\{}right)$$

$$w_j \{}leftarrow w_j - \{}eta \{}frac{\{}partial \{}mathcal{L}}{\{}partial w_j}, \{}quad b \{}leftarrow b - \{}eta \{}frac{\{}partial \{}mathcal{L}}{\{}partial b}$$

3.1.2 2. Regresión Logística

3.1.2.1 2.1 Concepto y Función Sigmoide

- **Definición:** Modelo paramétrico supervisado para **clasificación binaria** que predice la probabilidad $p = P(y = 1 \mid x) \in (0, 1)$.
- **Transformación Sigmoide / Logística:**

Para mapear el valor continuo lineal $z = x^\top w + b$ (denominado *logit*) al rango probabilístico $(0, 1)$, se aplica la función sigmoide $\sigma(z)$:

$$\{}hat{y} = \{}sigma(z) = \{}frac{1}{1 + e^{-z}}$$

- **Frontera de Decisión (*Decision Boundary*):**

El umbral estándar $P(y = 1 \mid x) = 0.5$ corresponde a $z = 0$ ($x^\top w + b = 0$), formando un hiperplano lineal de separación en el espacio de características.

3.1.2.2 2.2 Función de Pérdida (Entropía Cruzada Binaria / *Binary Cross-Entropy*)

Utilizar MSE sobre salidas sigmoideas genera una superficie de pérdida no convexa llena de zonas llanas (*plateaus*) donde el gradiente se desvanece ($\nabla \mathcal{L} \approx 0$). Por ello, se emplea la **Pérdida de Entropía Cruzada Binaria**:

$$\{}mathcal{L}(y, \{}hat{y}) = -\{}left[y \{}log(\{}hat{y}) + (1 - y) \{}log(1 - \{}hat{y}) \{}right]$$

- **Propiedades:** Es strictly convexa respecto a los parámetros (w, b), garantizando que el descenso de gradiente alcance el mínimo global sin quedar atrapado en mínimos locales subóptimos.

3.1.2.3 2.3 Visión Probabilística: Derivación desde la Distribución de Bernoulli

Al asumir que la etiqueta objetivo $y \in \{0, 1\}$ sigue una **distribución de Bernoulli** paramétrica con probabilidad de éxito $p = \hat{y} = \sigma(z)$:

$$P(y \mid x) = \hat{y}^y (1 - \hat{y})^{(1-y)}$$

La función de Entropía Cruzada equivale numéricamente al negativo de la función de Log-Verosimilitud (*Negative Log-Likelihood - NLL*). Por consiguiente, ajustar la Regresión Logística mediante Entropía Cruzada es equivalente a realizar la **Estimación de Máxima Verosimilitud (MLE)** de un proceso de Bernoulli.

3.1.3 3. Escalado de Características y Expansión Polinomial

3.1.3.1 3.1 Importancia del Escalado de Características (*Feature Scaling*)

Cuando las características poseen órdenes de magnitud desiguales, las curvas de nivel de la función de pérdida se deforman formando elipses alargadas. Esto provoca que el descenso de gradiente oscile en un patrón ineficiente en *zig-zag*, retrasando drásticamente la convergencia.

- **Estandarización (Standardization):**

$$x_j \mapsto \frac{x_j - \mu_j}{\sigma_j}$$

Transforma cada característica para que tenga media $\mu = 0$ y varianza unitaria $\sigma^2 = 1$, volviendo las curvas de nivel más circulares y estabilizando las actualizaciones de los gradientes.

3.1.3.2 3.2 Expansión Polinomial (*Polynomial Features*)

Cuando las clases no son separables por una frontera lineal recta, es posible mapear el espacio de características original a un espacio de mayor dimensión introduciendo combinaciones polinomiales.

- **Ejemplo para 2 características (x_1, x_2) a grado $d = 2$:**

$$z = w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_2^2 + w_5 x_1 x_2 + b$$

- **Ventajas e Inconvenientes:**
- Permite capturar límites de decisión no lineales complejas (elipses, parábolas) conservando la linealidad en los parámetros.
- Crecimiento exponencial del número de características al aumentar el grado d, elevando drásticamente el riesgo de sobreajuste (*overfitting*) y haciendo indispensable el escalado.

3.1.4 4. Regularización en Modelos Lineales (L_1 y L_2)

Para evitar que el modelo asigne pesos desproporcionadamente grandes a las características (previniendo el sobreajuste), se añade un término de penalización sobre los pesos a la función de pérdida:

$$\{}mathcal{J}(w, b) = \{}mathcal{L}(w, b) + \{}Omega(w)$$

| Regularización | Término de Penalización $\Omega(w)$ | Forma Geométrica | Comportamiento del Gradiente y Solución |

| :— | :— | :— | :— |

| L_2 **(Ridge)** | $\frac{\lambda}{2}\|w\|_2^2 = \frac{\lambda}{2}\sum_{j=1}^{d} w_j^2$ | Esfera / Círculo suave | Reduce la magnitud de todos los pesos proporcionalmente ($w \leftarrow w(1-\eta\lambda)$), pero **rara vez los hace exactamente cero**. |

| L_1 **(Lasso)** | $\lambda\|w\|_1 = \lambda\sum_{j=1}^{d} \|w_j\|$ | Diamante / Hiperoctaedro con esquinas en los ejes | Mantiene una magnitud de gradiente constante ($\lambda \cdot \text{signo}(w)$), **conduciendo pesos exactamente a cero** y generando soluciones esparsas (*feature selection*). |

3.1.5 5. Práctica y Aplicación en Código (PyTorch y Scikit-Learn)

```

import torch

import torch.nn as nn

import torch.optim as optim

from sklearn.linear_model import LogisticRegression

from sklearn.preprocessing import StandardScaler, PolynomialFeatures

from sklearn.pipeline import Pipeline



# ------------------------------------------------------------------------

# 1. Definición del Modelo de Regresión Logística en PyTorch

# ------------------------------------------------------------------------

class LogisticRegressionPyTorch(nn.Module):

    def __init__(self, input_dim):

        super(LogisticRegressionPyTorch, self).__init__()

        # Capa lineal que calcula el logit z = xW + b

        self.linear = nn.Linear(input_dim, 1)



    def forward(self, x):

        # Retorna el logit directamente.

        # BCEWithLogitsLoss aplica la sigmoide de forma numéricamente estable.

        return self.linear(x)


```

```
43
44  # ------------------------------------------------------------------------
45
46  # 2. Configuración y Bucle de Entrenamiento
47
48  # ------------------------------------------------------------------------
49
50  def train_logistic_regression(X_train_tensor, y_train_tensor, num_epochs=100, lr=0.01,
        weight_decay=1e-4):
51
52      device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
53
54
55
56      # Instanciación
57
58      input_dim = X_train_tensor.shape[1]
59
60      model = LogisticRegressionPyTorch(input_dim).to(device)
61
62
63
64      # Función de pérdida (Binary Cross Entropy con Sigmoide integrada)
65
66      criterion = nn.BCEWithLogitsLoss()
67
68
69
70      # Optimizador SGD con L2 Regularization (weight_decay)
71
72      optimizer = optim.SGD(model.parameters(), lr=lr, weight_decay=weight_decay)
73
74
75
76      X_train_tensor = X_train_tensor.to(device)
77
78      y_train_tensor = y_train_tensor.to(device).unsqueeze(1) # (B, 1)
79
80
81
82      for epoch in range(1, num_epochs + 1):
83
84          model.train()
85
86
87
88          # 1. Limpiar gradientes
89
90          optimizer.zero_grad()
91
92
93
94          # 2. Forward pass (logits)
95
96          logits = model(X_train_tensor)
97
98          loss = criterion(logits, y_train_tensor)
99
```

```
100
101
102         # 3. Backward pass y actualización de parámetros
103
104         loss.backward()
105
106         optimizer.step()
107
108
109
110         if epoch % 20 == 0 or epoch == 1:
111
112             print(f"Época [{epoch:3d}/{num_epochs}] | Loss: {loss.item():.4f}")
113
114
115
116     return model
117
118
119
120 # ------------------------------------------------------------------------
121
122 # 3. Implementación en Scikit-Learn
123
124 # ------------------------------------------------------------------------
125
126 def build_sklearn_logistic_pipeline(degree=2, penalty='l2', C=1.0):
127
128     """
129
130     Crea un pipeline completo con escalado, expansión polinomial y regresión
    logística.
131
132     Note: C en scikit-learn es la inversa de lambda (C = 1 / lambda).
133
134     """
135
136     pipeline = Pipeline([
137
138         ('scaler', StandardScaler()),
139
140         ('poly', PolynomialFeatures(degree=degree, include_bias=False)),
141
142         ('logistic', LogisticRegression(penalty=penalty, C=C, solver='saga',
    random_state=42))
143
144     ])
145
146     return pipeline
```

3.2 Conexión con las prácticas

La práctica recorre solución analítica, bucle de entrenamiento, convergencia, separación train/validation, superficie de pérdida y trayectoria del descenso de gradiente.

3.2.1 Fuente práctica: 03_01_Linear_regression.ipynb

Ruta: code/03_01_Linear_regression/03_01_Linear_regression.ipynb

Apartados detectados: 03_01 Linear regression; 1. Create a synthetic dataset (1D); 2. Try various values for (w, b); 3. Analytic solution; 4. Training loop.

Salidas gráficas almacenadas: 16.

Salida textual conservada:

> <Figure size 600x400 with 1 Axes> | <Figure size 600x400 with 1 Axes> | torch.Size([200, 2]) | tensor([[-1.0730],

Fragmento representativo:

```

import numpy as np
import torch
import matplotlib.pyplot as plt

np.random.seed(123)
torch.manual_seed(123)

w_true = 2.5
b_true = -1.0
noise_std = 0.6

n = 200

x = np.random.uniform(-3, 3, 200)

noise = np.random.normal(0, noise_std, n)

y = w_true*x + b_true+ noise

x_t = torch.tensor(x, dtype=torch.float32).view(-1,1)
y_t = torch.tensor(y, dtype=torch.float32).view(-1,1)

def plot_dataset(x_t, y_t):

    plt.figure(figsize=(6, 4))
    plt.scatter(x, y, alpha=0.7)

    plt.xlabel("x")
```

3.3 Autoevaluación razonada

Pregunta

¿Qué indica una curva de pérdida que deja de descender?

Solución. Que el paso puede ser demasiado grande, el entrenamiento puede haber convergido o existe un problema de escala/implementación.

Pregunta

¿Qué diferencia hay entre la solución analítica y SGD?

Solución. La primera resuelve una ecuación global bajo supuestos adecuados; SGD actualiza parámetros mediante gradientes de lotes o ejemplos.

3.4 Fuentes y trazabilidad

Diapositivas:

- `slides/03\_Linear\_regression.pdf`

Nota personal: apuntes/Apuntes del Tema 3 y Tema 4 - Modelos Lineales — Regresión Lineal y Regresión Logística.md

4 Regresión logística

Este capítulo se construye a partir de las fuentes de diapositivas y notebook indicadas al final.

4.1 Conexión con las prácticas

El notebook implementa la sigmoide y el entrenamiento, visualiza la frontera de decisión, añade features polinomiales y compara regularización L1/L2 mediante selección y validación cruzada.

4.1.1 Fuente práctica: `04_01_Logistic_regression.ipynb`

Ruta: code/04_01_Logistic_regression/04_01_Logistic_regression.ipynb

Apartados detectados: 04_01 Logistic regression; 1. Generate and visualize a synthetic dataset; 2. Implement logistic regression training loop; 3. Logistic regression with ‘torch.sigmoid‘, SGD optimizer, and built-in losses; 4. Visualize decision boundary on the training data.

Salidas gráficas almacenadas: 7.

Salida textual conservada:

> X shape: torch.Size([400, 2]) | y shape: torch.Size([400, 1]) | X_tr: torch.Size([320, 2]) | y_tr: torch.Size([320, 1])

Fragmento representativo:

```

import numpy as np
import torch
import matplotlib.pyplot as plt

# 1. Reproducibility
seed = 123
np.random.seed(seed)
torch.manual_seed(seed)

# 2. Dataset size
n_per_class = 200
n = 400

# 3. Define class distributions

mu_0 = torch.tensor([-1.0, 0.0])
cov_0 = torch.tensor([
    [0.8, 0.2],
    [0.2, 0.6]
```

```
])

mu_1 = torch.tensor([1.2, 0.6])
cov_1 = torch.tensor([
    [0.7, -0.25],
    [-0.25, 0.6]
])

dist_0 = torch.distributions.MultivariateNormal(mu_0, cov_0)
```

4.2 Autoevaluación razonada

Pregunta

¿Por qué la salida logística puede interpretarse como probabilidad?

Solución. La sigmoide transforma el logit real en un valor del intervalo (0,1), que se usa como probabilidad modelada.

Pregunta

¿Qué riesgo introduce un polinomio de grado alto?

Solución. Puede aumentar la flexibilidad y también el sobreajuste, por lo que se necesita regularización y validación.

4.3 Fuentes y trazabilidad

Diapositivas:

- `slides/04\_Logistic\_regression.pdf`

5 Perceptrones multicapa

5.1 Apuntes del Tema 5: Multilayer Perceptron (MLP I y II)

5.1.1 1. De Modelos Lineales a No Lineales

5.1.1.1 1.1 Limitaciones de la Transformación Afín

- **Modelos Lineales:** Modelos como la Regresión Lineal o la Regresión Logística calculan transformaciones afines de la forma $z = x^\top w + b$.
- **El Problema del Colapso Lineal:** Si se apilan múltiples capas compuestas únicamente por transformaciones afines, la red colapsa matemáticamente en un único modelo lineal equivalente.
- *Demostración:* Dadas dos capas lineales $H = XW^{(1)} + b^{(1)}$ y $O = HW^{(2)} + b^{(2)}$:

$$O = (X W^{(1)} + b^{(1)}) W^{(2)} + b^{(2)} = X (W^{(1)} W^{(2)}) + (b^{(1)} W^{(2)} + b^{(2)}) = X W + b$$

donde $W = W^{(1)}W^{(2)}$ y $b = b^{(1)}W^{(2)} + b^{(2)}$. La capacidad expresiva no aumenta con respecto a un modelo de una sola capa.

- **Separabilidad No Lineal (Problema XOR):** Las fronteras de decisión rectilíneas son incapaces de separar clases complejas o disjuntas (como la función lógica XOR o patrones circulares/concéntricos).

5.1.2 2. Capas Ocultas y Funciones de Activación

Para dotar al Perceptrón Multicapa (MLP) de la capacidad de aprender relaciones no lineales, se aplica una **función de activación no lineal** $\phi(\cdot)$ elemento a elemento tras cada transformación afín en las capas ocultas.

```
1
2 Entrada X (n x d)    [ Z = X·W(1) + b(1) ]    [ H =  (Z) ]    Salida O
```

5.1.2.1 2.1 Principales Funciones de Activación

Función	Ecuación Matemática $\phi(z)$	Rango de Salida	Propiedades y Derivada $\phi'(z)$
Sigmoide	$\sigma(z) = \frac{1}{1+e^{-z}}$	$(0, 1)$	Interpretación probabilística. **Problema:** Muestra zonas llanas en los extremos donde la derivada $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ tiende a 0, provocando desvanecimiento de gradiente (*vanishing gradient*).

|

| **Tanh** *(Tangente Hiperbólica)* | $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ | $(-1, 1)$ | Centrada en cero (facilita la optimización). Sufre de desvanecimiento de gradientes para $|z|$ elevados ($\tanh'(z) = 1 - \tanh^2(z)$). |

| **ReLU** *(Rectified Linear Unit)* | $\text{ReLU}(z) = \max(0, z)$ | $[0, \infty)$ | **Estándar en redes profundas:** Computacionalmente eficiente. Derivada $\phi'(z) = 1$ para $z > 0$ (evita el desvanecimiento de gradientes en la región positiva). **Riesgo:** *Dying ReLU* si $z \leq 0$ de forma permanente. |

5.1.3 3. Clasificación Multiclase: Regresión Softmax

Cuando el target y pertenece a una de las K categorías posibles ($K > 2$) sin un orden explícito, se emplea la codificación **One-Hot Encoding** ($y \in \mathbb{R}^K$).

5.1.3.1 3.1 Operador Softmax

La capa de salida genera K valores no acotados denominados **logits** ($O = [o_1, o_2, \ldots, o_K]$). Para transformar los logits en una distribución de probabilidad bien definida donde $\sum_{k=1}^{K} \hat{y}_k = 1$ y $\hat{y}_k > 0$, se aplica la función **Softmax**:

$$\hat{y}_k = \text{Softmax}(o_k) = \frac{e^{o_k}}{\sum_{j=1}^{K} e^{o_j}}$$

- **Propiedad Conservadora del Orden:** Preserva la ordenación monótona de los logits ($\arg\max_k \hat{y}_k = \arg\max_k o_k$).

5.1.3.2 3.2 Pérdida de Entropía Cruzada Categorical (*Categorical Cross-Entropy*)

Para una muestra con vector One-Hot y y predicciones $\hat{y}$:

$$\mathcal{L}(y, \hat{y}) = - \sum_{k=1}^{K} y_k \log(\hat{y}_k)$$

> **Nota de Implementación (PyTorch):** En la práctica, no se añade manualmente la capa `Softmax` al final del modelo. Se retorna el tensor de logits directamente y se utiliza `nn.CrossEntropyLoss()`, la cual combina internamente `Softmax` y `Cross-Entropy` de forma numéricamente estable utilizando el truco *log-sum-exp*.

5.1.4 4. Algoritmo de Propagación: Forward Pass y Backpropagation

5.1.4.1 4.1 Forward Pass (Propagación hacia Adelante)

Para un MLP de L capas con entrada $X \in \mathbb{R}^{n \times d}$ ($H^{(0)} = X$):

Para $i = 1, 2, \ldots, L-1$:

$$Z^{(i)} = H^{(i-1)} W^{(i)} + b^{(i)}$$

$$H^{(i)} = \phi(Z^{(i)})$$

Salida final (L):

$$O = H^{(L-1)} W^{(L)} + b^{(L)}$$

5.1.4.2 4.2 Backpropagation (Propagación hacia Atrás)

Consiste en la aplicación sistemática de la **Regla de la Cadena** multivariante del cálculo desde la función de pérdida $\mathcal{J}$ hasta las primeras capas de la red para obtener los gradientes $\frac{\partial \mathcal{J}}{\partial W^{(i)}}$ y $\frac{\partial \mathcal{J}}{\partial b^{(i)}}$:
$$\frac{\partial \mathcal{J}}{\partial W^{(i)}} = (H^{(i-1)})^\top \cdot \delta^{(i)}, \quad \text{donde } \delta^{(i)} = \frac{\partial \mathcal{J}}{\partial Z^{(i)}}$$

- **Gestión de Memoria:** Los tensores intermedios calculados durante el *forward pass* ($H^{(i)}, Z^{(i)}$) deben almacenarse en memoria para ser reevaluados durante el *backpropagation*, por lo que entrenar una red neuronal consume considerablemente más memoria que ejecutar inferencia (`with torch.no_grad():`).

5.1.5 5. Propiedades Teóricas e Inicialización de Parámetros

5.1.5.1 5.1 Teorema de Aproximación Universal

- **Definición:** Un Perceptrón Multicapa con tan solo **una capa oculta** y un número suficiente de neuronas no lineales puede aproximar cualquier función continua en un dominio compacto con un grado de precisión arbitrario $\epsilon > 0$.
- **¿Por qué usar redes profundas (más de 1 capa)?** El teorema demuestra la *existencia* teórica de dicha red, pero no especifica la cantidad de neuronas requeridas (que puede ser masiva/exponencial) ni si sus pesos se pueden aprender eficazmente con descenso de gradiente. Añadir profundidad permite representar funciones complejas mediante composiciones jerárquicas con un **número drásticamente menor de parámetros**.

5.1.5.2 5.2 Rompimiento de la Simetría e Inicialización

- **Inconveniente de Inicialización en Cero:** Si todos los pesos de una capa oculta se inicializan con la misma constante (p. ej. $w_{ij} = 0$), todas las neuronas de esa capa calcularán el mismo valor en el *forward pass* y recibirán exactamente el mismo gradiente en el *backward pass*. El MLP colapsaría al comportamiento de un modelo lineal único.
- **Estrategias de Inicialización:**
- **Xavier / Glorot Initialization:** Diseñada para activaciones sigmoideas o tanh. Mantiene constante la varianza de las activaciones y gradientes a través de las capas:

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$$

- **He / Kaiming Initialization:** Diseñada para activaciones **ReLU**, compensando que la mitad de las neuronas se desactivan (≤ 0):

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

5.1.6 6. Problemas en Redes Profundas: Desvanecimiento y Explosión del Gradiente

Al encadenar múltiples capas mediante la regla de la cadena, la magnitud del gradiente acumulado en las capas iniciales se calcula como un producto iterativo de matrices de pesos y derivadas de activación:

$$\{}frac{\{}partial \{}mathcal{J}}{\{}partial W^{(1)}} \{}propto \{}prod_{l=2}^{L} W^{(l)} \{}cdot \{}phi'(Z^{(l)})$$

- **Vanishing Gradients (Desvanecimiento):** Si las derivadas o los valores propios de los pesos son menores a 1 (p. ej., con sigmoide/tanh en zonas de saturación), el producto decrece exponencialmente hacia 0 conforme la red es más profunda. Las capas iniciales dejan de actualizarse y no aprenden.

- **Exploding Gradients (Explosión):** Si los valores de los pesos son mayores a 1, el producto crece exponencialmente provocando actualizaciones numéricamente inestables (`NaN` o `Inf`). Se mitiga aplicando **Gradient Clipping** (`nn.utils.clip_grad_norm_`).

5.1.7 7. Práctica y Aplicación en Código (PyTorch)

```

import torch

import torch.nn as nn

import torch.optim as optim



# ------------------------------------------------------------------------

# 1. Definición del Modelo MLP Multiclase

# ------------------------------------------------------------------------

class DeepMLP(nn.Module):

    def __init__(self, input_dim=8, hidden_dims=[12, 6], num_classes=3, dropout_rate
    =0.2):

        super(DeepMLP, self).__init__()



        layers = []

        in_dim = input_dim



        # Construcción de las capas ocultas con ReLU y Dropout

        for h_dim in hidden_dims:

            linear_layer = nn.Linear(in_dim, h_dim)

            # Inicialización He / Kaiming recomendada para ReLU

            nn.init.kaiming_normal_(linear_layer.weight, nonlinearity='relu')

            nn.init.zeros_(linear_layer.bias)

```

```
            layers.append(linear_layer)
            layers.append(nn.ReLU())
            layers.append(nn.Dropout(p=dropout_rate))
            in_dim = h_dim

        # Capa de salida lineal (entrega logits directos)
        out_layer = nn.Linear(in_dim, num_classes)
        nn.init.xavier_normal_(out_layer.weight)
        nn.init.zeros_(out_layer.bias)
        layers.append(out_layer)

        self.network = nn.Sequential(*layers)

    def forward(self, x):
        return self.network(x)

# ------------------------------------------------------------------------
# 2. Bucle de Entrenamiento
# ------------------------------------------------------------------------
def train_mlp_multiclass():
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Datos sintéticos: 100 muestras, 8 características, 3 clases
    X_dummy = torch.randn(100, 8)
    y_dummy = torch.randint(0, 3, (100,)) # Clases: 0, 1, 2

    model = DeepMLP(input_dim=8, hidden_dims=[12, 6], num_classes=3).to(device)

```

```
    # CrossEntropyLoss combina LogSoftmax + NLLLoss

    criterion = nn.CrossEntropyLoss()

    optimizer = optim.Adam(model.parameters(), lr=0.005)



    X_b = X_dummy.to(device)

    y_b = y_dummy.to(device)



    model.train()

    for epoch in range(1, 101):

        optimizer.zero_grad()



        # 1. Forward Pass (Logits)

        logits = model(X_b)

        loss = criterion(logits, y_b)



        # 2. Backward Pass

        loss.backward()



        # 3. Gradient Clipping opcional para evitar explosión

        nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)



        # 4. Actualización de parámetros

        optimizer.step()



        if epoch % 25 == 0:

            print(f"Época [{epoch:3d}/100] | Loss: {loss.item():.4f}")



if __name__ == "__main__":

    train_mlp_multiclass()
```

5.2 Conexión con las prácticas

Las prácticas muestran el paso de regresión logística multiclase a MLP y relacionan nn.Linear, nn.Sequential y autograd con el forward/backward pass.

5.2.1 Fuente práctica: 05_MLP_1.ipynb

Ruta: code/05_01_MLP/05_MLP_1.ipynb

Apartados detectados: Multi Logistic Regression.

Salidas gráficas almacenadas: 6.

Salida textual conservada:

> <Figure size 800x600 with 2 Axes> | Epoch [100/1000], Loss: 0.6977 | Epoch [200/1000], Loss: 0.6939 | Epoch [300/1000], Loss: 0.6931

Fragmento representativo:

```

import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import numpy as np
```

5.2.2 Fuente práctica: 05_02_MLP.ipynb

Ruta: code/05_02_MLP/05_02_MLP.ipynb

Apartados detectados: From 2 to many classes; Short Assignment.

Salidas gráficas almacenadas: 5.

Salida textual conservada:

> <torch._C.Generator at 0x774f2419fb10> | <Figure size 800x600 with 2 Axes> | Epoch [100/10000], Loss: 0.6439 | Epoch [200/10000], Loss: 0.6196

Fragmento representativo:

```

import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import make_circles, make_moons, make_blobs
from sklearn.model_selection import train_test_split

import matplotlib.pyplot as plt
import numpy as np
```

5.2.3 Fuente práctica: 05_04_MLP.ipynb

Ruta: code/05_04_MLP/05_04_MLP.ipynb

Apartados detectados: A more compact way to write our code; nn.Linear; nn.Sequential; Assignment.

Salidas gráficas almacenadas: 6.

Salida textual conservada:

> <Figure size 640x480 with 1 Axes> | Tipo de X_train: <class 'torch.Tensor'> torch.float32 | Tipo de y_train: <class 'torch.Tensor'> torch.float32 | Epoch [100/1200], Loss: 0.7300

Fragmento representativo:

```

class HiddenLayer(nn.Module):
  def __init__(self, input_size, hidden_size):
    super(HiddenLayer, self).__init__()
    self.W = nn.Parameter(torch.randn(input_size, hidden_size))
    self.b = nn.Parameter(torch.zeros(1, hidden_size))

  def forward(self, X):
    return torch.matmul(X, self.W) + self.b
```

5.2.4 Fuente práctica: 02_01_BasicPytorch.ipynb

Ruta: code/Practice_02/02_01_BasicPytorch.ipynb

Apartados detectados: Mathematical Foundations. Simple Use of PyTorch; Tensors; Indexing and Slicing; Operations and Basic Properties of Tensor Arithmetic; Broadcasting.

Salidas gráficas almacenadas: 0.

Fragmento representativo:

```

x = torch.tensor([1.0, 2, 4, 8])
y = torch.tensor([2, 2, 2, 2])
x + y, x - y, x * y, x / y, x ** y
```

5.2.5 Fuente práctica: 02_02_Autograd.ipynb

Ruta: code/Practice_02/02_02_Autograd.ipynb

Apartados detectados: Mathematical Foundations. Automatic differentiation with PyTorch; Mathematical Foundations. Automatic differentiation with TensorFlow; Summary: Key Differences Between PyTorch and TensorFlow for autograd.

Salidas gráficas almacenadas: 0.

Fragmento representativo:

```

a = torch.tensor(3.0, requires_grad=True)
b = torch.tensor(4.0, requires_grad=True)
```

```
c = torch.tensor(5.0, requires_grad=True)

print(f"Initial state:")
print(f"a = {a}, a.grad = {a.grad}")
print(f"b = {b}, b.grad = {b.grad}")
print(f"c = {c}, c.grad = {c.grad}\n")
```

5.3 Autoevaluación razonada

Pregunta

¿Qué representa un logit en clasificación multiclase?

Solución. La puntuación sin normalizar de una clase antes de aplicar la transformación usada por la pérdida o la probabilidad.

Pregunta

¿Qué calcula autograd?

Solución. Construye y recorre el grafo de operaciones para obtener derivadas respecto de los tensores que requieren gradiente.

5.4 Fuentes y trazabilidad

Diapositivas:

- `slides/05\_01\_MLP.pdf`
- `slides/05\_03\_MLP.pdf`
- `slides/05\_NotesPytorch1.pdf`

Nota personal: apuntes/Apuntes del Tema 5 - Multilayer Perceptron (MLP I y II).md

6 Regularización en redes profundas

6.1 Apuntes del Tema 6: Regularización en Redes Profundas

6.1.1 1. El Problema del Sobreajuste (*Overfitting*) en Redes Profundas

6.1.1.1 1.1 Naturaleza del Problema

- **Alta Capacidad Expresiva:** Las redes neuronales profundas poseen millones de parámetros libres. Esta gran capacidad les permite memorizar no solo los patrones subyacentes de los datos, sino también el ruido de entrenamiento.
- **Brecha de Generalización (*Generalization Gap*):** Se define como la diferencia entre la pérdida/error en el conjunto de validación (o prueba) y el conjunto de entrenamiento.
- *Baja varianza / Buen ajuste:* La pérdida de validación disminuye a la par con la de entrenamiento.
- *Alta varianza / Overfitting:* A partir de cierto punto, la pérdida de entrenamiento sigue bajando mientras la pérdida de validación empieza a crecer, ensanchando la brecha de generalización.
- **Objetivo de la Regularización:** Modificar o restringir el proceso de aprendizaje para reducir el error de prueba/generalización sin incrementar excesivamente el error de entrenamiento.

6.1.2 2. Decaimiento de Pesos (*Weight Decay* / Regularización L_2 vs. L_1)

La forma clásica de regularización consiste en añadir un término de penalización $\Omega(W)$ a la función de pérdida original $\mathcal{L}(W)$:

$$\mathcal{J}(W) = \mathcal{L}(W) + \lambda \Omega(W)$$

donde $\lambda > 0$ es el hiperparámetro que controla la intensidad de la regularización.

6.1.2.1 2.1 Decaimiento de Pesos (L_2 Regularization)

- **Penalización:** Se utiliza la norma L_2 al cuadrado de las matrices de pesos:

$$\Omega(W) = \frac{1}{2} \Vert W \Vert_2^2 = \frac{1}{2} \sum_{i} \sum_{j} w_{ij}^2$$

- **Efecto en el Gradiente y Actualización (SGD):**

$$\{}frac{\{}partial \{}mathcal{J}}{\{}partial W} = \{}frac{\{}partial \{}mathcal{L}}{\{}partial W} + \{}lambda W$$

$$W \{}leftarrow W - \{}eta \{}frac{\{}partial \{}mathcal{J}}{\{}partial W} = W(1 - \{}eta \{}lambda) - \{}eta \{}frac{\{}partial \{}mathcal{L}}{\{}partial W}$$

- **Interpretación:** Antes de aplicar el paso de gradiente habitual, los pesos se reducen por un factor multiplicativo $(1 - \eta\lambda) < 1$. Esto fuerza a la red a mantener pesos pequeños y distribuidos de forma suave, previniendo comportamientos extremos frente a pequeñas variaciones en la entrada.

6.1.2.2 2.2 Regularización L_1 (Lasso)

- **Penalización:** Suma de los valores absolutos de los pesos:

$$\{}Omega(W) = \{}Vert{}W\{}Vert{}_1 = \{}sum_{i} \{}sum_{j} \{}vert{}w_{ij}\{}vert{}$$

- **Efecto:** Mantiene una magnitud de penalización constante $\lambda \cdot \text{signo}(W)$ independientemente del tamaño del peso, lo que empuja muchos pesos **exactamente a cero**, generando matrices de pesos esparsas (*sparse models*) y actuando como selección implícita de características.

6.1.3 3. Dropout (Inyección de Ruido en Activaciones)

Proposed por Srivastava et al. (2014), es una de las técnicas de regularización más efectivas en redes profundas.

6.1.3.1 3.1 Mecanismo de Funcionamiento

Durante el entrenamiento, en cada iteración o minilote, cada neurona oculta se "apaga" (se fuerza a cero) con una probabilidad $p \in (0, 1)$ elegida de forma independiente.

- **Formulación Matemática (Inverted Dropout):**

Dada una activación intermedia h, se transforma en h' mediante:

$$h' = \{}begin{cases} 0 & \{}text{con probabilidad } p \{}\{} \{}frac{h}{1-p} & \{}text{con probabilidad } 1-p \{}end{cases}$$

- **Reescalado ($\frac{1}{1-p}$):** Escalar por $\frac{1}{1-p}$ durante el entrenamiento garantiza que el valor esperado de la activación sea constante ($E[h'] = h$), evitando modificar la magnitud de las entradas en la fase de test.

```
Antes de Dropout                Después de Dropout (con p = 0.4)

  [x1]  [x2]                      [x1]  [x2]


  [h1]  [h2]                      [ 0 ]  [h2]   <- Neurona h1 desactivada


  [o1]  [o2]                      [o1]  [o2]
```

6.1.3.2 3.2 ¿Por qué funciona Dropout?

1. **Prevención de la Co-adaptación:** Fuerza a cada neurona a aprender características útiles por sí misma, en lugar de depender de la presencia corregidora de otras neuronas específicas.

1. **Ensamble Implícito (*Implicit Ensembling*):** Entrenar con Dropout equivale a entrenar en cada paso un subconjunto distinto de la red (*thinned network*) que comparte pesos. La evaluación final sin Dropout equivale geométricamente a promediar las predicciones de un número masivo de subredes.

1. **Inferencia / Test:** Durante la fase de evaluación, el Dropout debe **desactivarse** llamando a `model.eval()` en PyTorch.

6.1.4 4. Parada Temprana (*Early Stopping*)

Es una técnica de regularización no paramétrica que aprovecha la dinámica temporal del entrenamiento.

6.1.4.1 4.1 Criterio de Funcionamiento

- **Fundamento:** A medida que aumentan las épocas de entrenamiento, el error de entrenamiento disminuye de manera monótona, pero el error de validación alcanza un mínimo y luego empieza a aumentar.

- **Mecanismo:**

1. Se evalúa el error sobre el conjunto de validación al final de cada época.

1. Se guarda una copia de los mejores pesos del modelo cuando la pérdida de validación alcanza un nuevo mínimo.
2. **Criterio de Paciencia (*Patience*):** Si la pérdida de validación no mejora durante un número P determinado de épocas consecutivas, el entrenamiento se interrumpe y se restauran los pesos óptimos guardados.

```
1
2  Pérdida
3
4            \         /  <- Pérdida de Validación (Empieza a subir)
5
6             \_______/
7
8                Parada Temprana (Guardar mejor modelo)
9
10              \
11
12               \_______ <- Pérdida de Entrenamiento (Sigue bajando)
13
14                    Épocas
```

6.1.5 5. Práctica y Aplicación en Código (PyTorch)

A continuación se presenta la implementación modular que integra **Weight Decay** (L_2), **Dropout** y **Early Stopping** utilizando PyTorch:

```

import torch

import torch.nn as nn

import torch.optim as optim

from torch.utils.data import DataLoader, TensorDataset



# ------------------------------------------------------------------------

# 1. Definición del Modelo con Dropout

# ------------------------------------------------------------------------

class RegularizedMLP(nn.Module):

    def __init__(self, input_dim=20, hidden_dim=64, num_classes=2, dropout_p=0.5):

        super(RegularizedMLP, self).__init__()



        self.network = nn.Sequential(

            nn.Linear(input_dim, hidden_dim),

            nn.ReLU(),

            nn.Dropout(p=dropout_p),  # Inyección de ruido en activaciones



            nn.Linear(hidden_dim, hidden_dim),

            nn.ReLU(),

            nn.Dropout(p=dropout_p),



            nn.Linear(hidden_dim, num_classes)  # Logits de salida

        )



    def forward(self, x):

        return self.network(x)

```

```
54
55
56  # ------------------------------------------------------------------------
57
58  # 2. Clase para Gestionar Early Stopping
59
60  # ------------------------------------------------------------------------
61
62  class EarlyStopping:
63
64      def __init__(self, patience=5, delta=0.001):
65
66          self.patience = patience
67
68          self.delta = delta
69
70          self.best_loss = float('inf')
71
72          self.counter = 0
73
74          self.early_stop = False
75
76          self.best_state_dict = None
77
78
79
80      def __call__(self, val_loss, model):
81
82          if val_loss < self.best_loss - self.delta:
83
84              self.best_loss = val_loss
85
86              self.counter = 0
87
88              self.best_state_dict = model.state_dict().copy() # Guardar mejores pesos
89
90          else:
91
92              self.counter += 1
93
94              if self.counter >= self.patience:
95
96                  self.early_stop = True
97
98
99
100 # ------------------------------------------------------------------------
101
102 # 3. Bucle de Entrenamiento y Validación
103
104 # ------------------------------------------------------------------------
105
106 def train_with_regularization():
107
108     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
109
110
111
```

```
    # Datos sintéticos

    X_train = torch.randn(800, 20)

    y_train = torch.randint(0, 2, (800,))

    X_val = torch.randn(200, 20)

    y_val = torch.randint(0, 2, (200,))



    train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=32, shuffle
    =True)

    val_loader = DataLoader(TensorDataset(X_val, y_val), batch_size=32, shuffle=False
    )



    model = RegularizedMLP(input_dim=20, dropout_p=0.3).to(device)

    criterion = nn.CrossEntropyLoss()



    # Regularización L2 (Weight Decay) aplicada directamente en el optimizador

    optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)

    early_stopper = EarlyStopping(patience=10)



    epochs = 100

    for epoch in range(1, epochs + 1):

        # FASE DE ENTRENAMIENTO (Dropout Activo)

        model.train()

        train_loss = 0.0

        for images, labels in train_loader:

            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()

            outputs = model(images)

            loss = criterion(outputs, labels)

            loss.backward()

            optimizer.step()

```

```
            train_loss += loss.item() * images.size(0)


        train_loss /= len(train_loader.dataset)


        # FASE DE VALIDACIÓN (Dropout Desactivado)
        model.eval()  # Importante: desactiva Dropout
        val_loss = 0.0
        with torch.no_grad():
            for images, labels in val_loader:
                images, labels = images.to(device), labels.to(device)
                outputs = model(images)
                loss = criterion(outputs, labels)
                val_loss += loss.item() * images.size(0)


        val_loss /= len(val_loader.dataset)


        print(f"Época [{epoch:3d}/{epochs}] | Train Loss: {train_loss:.4f} | Val Loss:
    {val_loss:.4f}")


        # Comprobar criterio de Parada Temprana
        early_stopper(val_loss, model)
        if early_stopper.early_stop:
            print(f"\n[Early Stopping] Entrenamiento detenido en la época {epoch}.")
            # Restaurar mejores pesos
            model.load_state_dict(early_stopper.best_state_dict)
            break


if __name__ == '__main__':
    train_with_regularization()
```

6.2 Conexión con las prácticas

La práctica permite relacionar las curvas de entrenamiento/validación con weight decay, dropout y early stopping. La interpretación se limita a las métricas y figuras guardadas en el notebook.

6.2.1 Fuente práctica: `06_Regularization_01.ipynb`

Ruta: code/06_02/06_Regularization_01.ipynb

Apartados detectados: Assignment.

Salidas gráficas almacenadas: 13.

Salida textual conservada:

> Training model: 100%| | 5000/5000 [00:13<00:00, 363.98epoch/s, Train Loss=0.0005, Test Loss=1.5976, Train Acc=100.0%, Test Acc=79.5%] | Training completed in 13.74 seconds. | <Figure size 1200x400 with 2 Axes> | <Figure size 1200x400 with 2 Axes>

Fragmento representativo:

```

import torch.nn as nn
import torch
import torch.optim as optim
from tqdm import tqdm
import matplotlib.pyplot as plt
import time
import numpy as np

fashion_mnist = torch.load('fashion_mnist.pt')
X_train = fashion_mnist['X_train']
y_train = fashion_mnist['y_train']
X_test = fashion_mnist['X_test']
y_test = fashion_mnist['y_test']
```

6.3 Autoevaluación razonada

Pregunta

¿Qué síntoma sugiere sobreajuste?

Solución. La pérdida de entrenamiento sigue mejorando mientras la validación empeora o deja de mejorar.

Pregunta

¿Qué hace early stopping?

Solución. Detiene el entrenamiento cuando la métrica de validación no mejora según un criterio y puede restaurar el mejor estado.

6.4 Fuentes y trazabilidad

Diapositivas:

- `slides/06\_01.pdf`

Nota personal: apuntes/Apuntes del Tema 6 - Regularización en Redes Profundas.md

7 Descenso de gradiente y optimizadores

7.1 Apuntes del Tema 8: Descenso de Gradiente, Métodos Acelerados y Adaptativos

7.1.1 1. Fundamentos de Optimización en Deep Learning

7.1.1.1 1.1 Optimización Matemática vs. Aprendizaje Profundo

- **Optimización Pura:** Su objetivo principal es minimizar una función objetivo $\mathcal{L}(w)$ sobre un conjunto de datos de entrenamiento para reducir el error de entrenamiento.
- **Deep Learning:** Su verdadero objetivo es reducir el **error de generalización** (desempeño en datos no vistos del conjunto de prueba), combinando la minimización del error con técnicas para mitigar el sobreajuste (*overfitting*).

7.1.1.2 1.2 Retos en la Superficie de Pérdida

1. **Mínimos Locales:** Puntos donde el gradiente es cero ($\nabla f(x) = 0$) y la matriz Hessiana es definida positiva. En redes profundas con alta dimensionalidad, la mayoría de los puntos críticos no son mínimos locales, sino puntos de silla.

1. **Puntos de Silla (*Saddle Points*):** Puntos donde el gradiente se anula, pero la matriz Hessiana posee autovalores tanto positivos como negativos (la función sube en algunas direcciones y baja en otras).

1. **Desvanecimiento del Gradiente (*Vanishing Gradients*):** Regiones extremadamente llanas de la superficie de pérdida (por ejemplo, en las zonas de saturación de $\tanh(x)$ o la sigmoide) donde $\|\nabla f(x)\| \approx 0$, ralentizando drásticamente la actualización de los parámetros.

```
Superficie de Pérdida

          /\

         /  \   <- Máximo Local

    ____/    \____

```

```
    /              \  <- Punto de Silla (Hessiana con autovalores mixtos)

   /                \____

                Mínimo Local / Global
```

7.1.1.3 1.3 Convexidad y Series de Taylor

- **Convexidad:** Un conjunto $\mathcal{X}$ es convexo si el segmento que une cualquier par de puntos $a, b \in \mathcal{X}$ está totalmente contenido en $\mathcal{X}$. Una función f es convexa si:

$$\lambda f(x) + (1-\lambda)f(x') \ge f(\lambda x + (1-\lambda)x'), \quad \forall \lambda \in [0, 1]$$

- *Propiedad clave:* En funciones estrictamente conexas, todo mínimo local es también un **mínimo global**. Aunque las funciones de pérdida en redes profundas son no convexas, exhiben comportamiento convexo en las vecindades inmediatas de los mínimos locales.

- **Expansión en Serie de Taylor:** Aproximación polinomial local de una función diferenciable $f(x)$ alrededor de un punto x_0:

$$f(x) \approx f(x_0) + \nabla f(x_0)^\top (x - x_0) + \frac{1}{2}(x - x_0)^\top \mathbf{H}(x - x_0)$$

donde $\mathbf{H}$ es la matriz Hessiana de segundas derivadas parciales ($\mathbf{H}_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$).

7.1.2 2. Algoritmos de Descenso de Gradiente

7.1.2.1 2.1 Descenso de Gradiente Unidimensional y Multivariante

Partiendo de la expansión de Taylor de primer orden $f(x+\epsilon) \approx f(x)+\epsilon\nabla f(x)$, para garantizar la máxima disminución del valor de f, debemos movernos en la dirección opuesta al gradiente ($\epsilon = -\eta\nabla f(x)$ con $\eta > 0$):

$$w_{t+1} = w_t - \eta \nabla f(w_t)$$

- **Impacto de la Tasa de Aprendizaje (η):**
- Si η es demasiado pequeño: Convergencia extremadamente lenta.

- Si η es demasiado grande: Oscilaciones inestables e incapacidad para converger o divergencia.

7.1.2.2 2.2 Variantes del Descenso de Gradiente según el Tamaño del Lote

| Variante | Tamaño del Lote (*Batch Size*) | Ventajas | Inconvenientes |

| — | — | — | — |

| **Batch Gradient Descent** | Muestras totales N | Cálculo exacto del gradiente y convergencia suave. | Inviable en memoria RAM/GPU para datasets grandes; computacionalmente lento.

|

| **Stochastic Gradient Descent (SGD)** | 1 muestra ($|B| = 1$) | Actualizaciones muy rápidas; la varianza del estimador ayuda a escapar de mínimos locales.

| Gradientes con altísima varianza, trayectoria en "zig-zag" muy ruidosa.

|

| **Mini-Batch SGD** | Lote pequeño ($|B| \in [32, 512]$) | **Estándar en la práctica:** Aprovecha la paralelizació en GPU/vectorización y suaviza el gradiente.

| Requiere afinar la tasa de aprendizaje global η.

|

7.1.2.3 2.3 Método de Newton (Segundo Orden)

Aprovecha la curvatura local mediante la matriz Hessiana para calcular el paso óptimo:

$$\{}Delta x = -\{}mathbf{H}^{-1} \{}nabla f(x)$$

- **Ventaja:** Ajusta automáticamente la amplitud del paso sin depender fuertemente de η.
- **Desventaja:** Computacionalmente prohibitivo en Deep Learning ($O(d^3)$ para invertir $\mathbf{H}$ con millones de parámetros) y sensible a direcciones donde la Hessiana es definida negativa.

7.1.3 3. Optimizadores Acelerados y Adaptativos

```

Evolución de Optimizadores:

SGD    Momentum (Inercia v_t)    AdaGrad (Escalado por g²)    RMSProp (Promedio Móvil
    )    Adam (Momentum + RMSProp)
```

7.1.3.1 3.1 Método del Momento (*Momentum*)

Introduce una variable de velocidad v_t que acumula los gradientes pasados como un promedio móvil ponderado exponencialmente (física de una bola rodando cuesta abajo):

$$v_t \{}leftarrow \{}beta v_{t-1} + g_t$$

$$w_t \{}leftarrow w_{t-1} - \{}eta v_t$$

- **Propiedades:** Amortigua las oscilaciones en direcciones de alta varianza y acelera el avance a lo largo de barrancos de la superficie de pérdida. Típicamente $\beta \approx 0.9$.

7.1.3.2 3.2 AdaGrad (*Adaptive Gradient Algorithm*)

Adapta la tasa de aprendizaje de cada parámetro individualmente basándose en la suma acumulada de sus gradientes al cuadrado s_t:

$$s_t \{}leftarrow s_{t-1} + g_t \{}odot g_t$$

$$w_t \{}leftarrow w_{t-1} - \{}frac{\{}eta}{\{}sqrt{s_t + \{}epsilon}} \{}odot g_t$$

- **Propiedades:** Parámetros con actualizaciones frecuentes o grandes reciben tasas de aprendizaje efectivas más pequeñas, mientras que parámetros con actualizaciones raras o pequeñas reciben pasos más grandes.

- **Inconveniente:** s_t crece de forma monótona, haciendo que la tasa de aprendizaje efectiva tienda a cero y detenga el aprendizaje prematuramente.

7.1.3.3 3.3 RMSProp y AdaDelta

Resuelven la acumulación desmedida de AdaGrad sustituyendo la suma por un promedio móvil con decaimiento exponencial (*leaky average*) con parámetro $\gamma \in (0, 1)$:

$$s_t \leftarrow \gamma s_{t-1} + (1 - \gamma) g_t \odot g_t$$

$$w_t \leftarrow w_{t-1} - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot g_t$$

7.1.3.4 3.4 Adam (*Adaptive Moment Estimation*)

El optimizador más popular y robusto en la práctica. Combina los beneficios del **Momento** (estimación del primer momento / media v_t) y de **RMSProp** (estimación del segundo momento / varianza s_t), añadiendo una **corrección de sesgo** para evitar la deriva hacia cero durante las primeras iteraciones:

1. **Primer Momento (Momento):**

$$v_t \leftarrow \beta_1 v_{t-1} + (1 - \beta_1) g_t$$

1. **Segundo Momento (RMSProp):**

$$s_t \leftarrow \beta_2 s_{t-1} + (1 - \beta_2) (g_t \odot g_t)$$

1. **Corrección de Sesgo (*Bias Correction*):**

$$\hat{v}_t = \frac{v_t}{1 - \beta_1^t}, \quad \hat{s}_t = \frac{s_t}{1 - \beta_2^t}$$

1. **Actualización de Parámetros:**

$$w_t \leftarrow w_{t-1} - \frac{\eta}{\sqrt{\hat{s}_t} + \epsilon} \odot \hat{v}_t$$

Hiperparámetros por defecto estándar: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

7.1.4 4. Polítas de Ajuste (*Learning Rate Schedulers*) y Warmup

7.1.4.1 4.1 Planificadores de Tasa de Aprendizaje (*Schedulers*)

Reducen la tasa de aprendizaje η a medida que avanza el entrenamiento para evitar sobrepasar el mínimo local y permitir un ajuste fino:

- **Polynomial Decay:** $\eta_t = \eta_0 \left(1 - \frac{t}{T}\right)^p$.

- **Step / Factor Decay:** $\eta_{t+1} = \eta_t \cdot \alpha$ con $\alpha \in (0, 1)$ cada k épocas.
- **Cosine Annealing:** Modula la tasa siguiendo una curva cosenoidal decreciente.
- **ReduceLROnPlateau:** Reduce η por un factor cuando una métrica de validación deja de mejorar durante varias épocas.

7.1.4.2 4.2 Calentamiento (*Warmup*)

Consiste en iniciar con una tasa de aprendizaje muy pequeña durante las primeras épocas e incrementarla linealmente hasta el valor η_0 antes de aplicar cualquier estrategia de decaimiento. Previene la desestabilización inicial de los pesos cuando los gradientes son excesivamente volátiles.

7.1.5 5. Práctica y Aplicación en Código (PyTorch)

A continuación se incluye una implementación completa en PyTorch que compara el uso de **SGD con Momentum**, **Adam** y un **Scheduler con Warmup**:

```

import torch

import torch.nn as nn

import torch.optim as optim

from torch.optim.lr_scheduler import LambdaLR, ReduceLROnPlateau



# ------------------------------------------------------------------------

# 1. Definición de una Red de Prueba

# ------------------------------------------------------------------------

class DummyNet(nn.Module):

    def __init__(self, input_dim=10, num_classes=2):

        super(DummyNet, self).__init__()

        self.fc = nn.Sequential(

            nn.Linear(input_dim, 32),

            nn.ReLU(),

            nn.Linear(32, num_classes)

        )



    def forward(self, x):

```

```
        return self.fc(x)


# ------------------------------------------------------------------------
# 2. Configuración de Optimizadores en PyTorch
# ------------------------------------------------------------------------
def setup_optimizers(model):
    # a) SGD con Momentum y Weight Decay
    optimizer_sgd = optim.SGD(
        model.parameters(),
        lr=0.01,
        momentum=0.9,
        weight_decay=1e-4
    )

    # b) Adam (Adaptive Moment Estimation)
    optimizer_adam = optim.Adam(
        model.parameters(),
        lr=0.001,
        betas=(0.9, 0.999),
        eps=1e-8
    )

    return optimizer_sgd, optimizer_adam


# ------------------------------------------------------------------------
# 3. Implementación de Warmup + Cosine Decay Scheduler
# ------------------------------------------------------------------------
def get_warmup_scheduler(optimizer, warmup_epochs=5, max_epochs=50):
    def lr_lambda(epoch):

```

```
        if epoch < warmup_epochs:

            # Incremento lineal durante el warmup

            return float(epoch + 1) / float(warmup_epochs)

        # Decaimiento pasados los epochs de warmup

        return 0.5 * (1.0 + torch.cos(torch.tensor((epoch - warmup_epochs) / (
    max_epochs - warmup_epochs) * 3.14159)))


    return LambdaLR(optimizer, lr_lambda=lr_lambda)


# -----------------------------------------------------------------------------

# 4. Ejemplo de Bucle de Entrenamiento

# -----------------------------------------------------------------------------

def train_demo():

    model = DummyNet()

    optimizer = optim.Adam(model.parameters(), lr=1e-3)

    scheduler = get_warmup_scheduler(optimizer, warmup_epochs=5, max_epochs=20)

    criterion = nn.CrossEntropyLoss()


    # Entrada y etiquetas de prueba

    inputs = torch.randn(16, 10)

    labels = torch.randint(0, 2, (16,))


    print("--- Demostración de Entrenamiento con Scheduler ---")

    for epoch in range(1, 21):

        optimizer.zero_grad()

        outputs = model(inputs)

        loss = criterion(outputs, labels)

        loss.backward()

        optimizer.step()


```

```
153
154         # Obtener LR actual
155
156         current_lr = optimizer.param_groups[0]['lr']
157
158         print(f"Época [{epoch:2d}/20] | Loss: {loss.item():.4f} | LR: {current_lr:.6f}
    ")
159
160
161
162         # Paso del scheduler al final de la época
163
164         scheduler.step()
165
166
167
168 if __name__ == "__main__":
169
170     train_demo()
```

¿Hay algún punto o demostración de este bloque de optimizadores sobre el que quieras profundizar o tienes alguna duda específica?

7.2 Conexión con las prácticas

Las prácticas comparan optimizadores y schedulers en experimentos con datos y pérdidas observables. Las curvas incluidas en las salidas permiten discutir velocidad, estabilidad y sensibilidad al learning rate.

7.2.1 Fuente práctica: 08_01_GradientDescent_Lab.ipynb

Ruta: code/08_01_GradientDescent/08_01_GradientDescent_Lab.ipynb

Apartados detectados: Experiments.

Salidas gráficas almacenadas: 2.

Salida textual conservada:

> <matplotlib.collections.PathCollection at 0x727622433ed0> | <Figure size 640x480 with 1 Axes> | — Running Experiment: SGD (batch_size = 1) — | Training Model: 100%| | 20/20 [00:26<00:00, 1.34s/epoch, Train Loss=0.5167, Val Loss=0.5170, Val Acc=0.8775, LR=0.000010]

Fragmento representativo:

```
1
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 from sklearn.datasets import make_moons
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import StandardScaler
8 import matplotlib.pyplot as plt
9 import time
```

7.2.2 Fuente práctica: 08_02_Optimizers_Lab.ipynb

Ruta: code/08_01_GradientDescent/08_02_Optimizers_Lab.ipynb

Salidas gráficas almacenadas: 2.

Salida textual conservada:

> <matplotlib.contour.QuadContourSet at 0x7eec04143cb0> | <Figure size 800x600 with 1 Axes> | <Figure size 1600x600 with 2 Axes> | Optimizer | Final Point (x, y) | Final Loss

Fragmento representativo:

```

import torch
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import LogNorm
```

7.2.3 Fuente práctica: 08_03_ExploringTorchOptim.ipynb

Ruta: code/08_01_GradientDescent/08_03_ExploringTorchOptim.ipynb

Apartados detectados: Dataset 1; Dataset 2.

Salidas gráficas almacenadas: 8.

Salida textual conservada:

> <Figure size 1800x500 with 3 Axes> | Entrenando: SGD | Constant LR... | Entrenando: SGD | Polynomial decay... | Entrenando: SGD | Multifactor scheduler...

Fragmento representativo:

```

import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from torch.utils.data import DataLoader, TensorDataset
```

7.2.4 Fuente práctica: 02_03_AutogradExercise.ipynb

Ruta: code/Practice_02/02_03_AutogradExercise.ipynb

Apartados detectados: Mathematical Foundations. Automatic differentiation with PyTorch. Exercise.

Salidas gráficas almacenadas: 0.

Salida textual conservada:

> x = 1.0, y = 2.0 | f(x, y) = 6.0 | df/dx = 12.0 | df/dy = 3.0

Fragmento representativo:

```

import torch


def f(x, y):
    n = int(torch.floor(torch.abs(y)).item()) + 1
    if y > 0:
        c =  sum(x**k * y for k in range(1, n + 1))
    else:
        c = torch.log(1 + x**2 + y**2)
    return c
```

7.2.5 Fuente práctica: 02_04_Autograd_Lab.ipynb

Ruta: code/Practice_02/02_04_Autograd_Lab.ipynb

Apartados detectados: Optimization with autograd.

Salidas gráficas almacenadas: 1.

Salida textual conservada:

> <Figure size 800x600 with 1 Axes> | tensor(6968273.5000, grad_fn=<AddBackward0>) | tensor(8966329., grad_fn=<AddBackward0>)

Fragmento representativo:

```

c = torch.tensor([[0.0, 10.0], [1.0, 1.0], [12.0, 2.0], [15.0, 11.0], [8.0, 15.0],
    [5.0, 7.0]], requires_grad=True)
w = torch.tensor([250000, 100000, 80000, 400000, 1500, 30000])
```

7.3 Autoevaluación razonada

Pregunta

¿Qué problema intenta aliviar momentum?

Solución. La oscilación y el avance lento en direcciones con curvaturas o escalas muy diferentes.

Pregunta

¿Por qué un scheduler modifica el learning rate?

Solución. Para adaptar el tamaño de las actualizaciones durante el entrenamiento y favorecer convergencia o refinamiento final.

7.4 Fuentes y trazabilidad

Diapositivas:

- slides/08_01_Optimizers.pdf

- `slides/08\_02\_Optimizers2.pdf`

Nota personal: apuntes/Apuntes del Tema 8 - Descenso de Gradiente, Métodos Acelerados y Adaptativos.md

8 Redes neuronales convolucionales

8.1 Apuntes del Tema 9: Redes Neuronales Convolucionales (CNN I y II)

8.1.1 1. De MLPs a Convoluciones: Motivación e Sesgos Inductivos

8.1.1.1 1.1 Limitaciones del Perceptrón Multicapa (MLP) en Imágenes

- **Explosión de Parámetros:** Un MLP requiere aplanar la imagen de dimensión $C \times H \times W$ a un vector unidimensional $x \in \mathbb{R}^{C \cdot H \cdot W}$.
- *Ejemplo:* Para una imagen pequeña de $64 \times 64 \times 3 = 12\,288$ entradas, una primera capa oculta con $1\,024$ neuronas requiere $(12\,288 \times 1\,024) + 1\,024 \approx 12.58$ millones de parámetros.
- **Pérdida de Estructura Espacial:** Al aplanar la matriz de píxeles, el MLP trata la distancia entre píxeles adyacentes de la misma manera que la distancia entre píxeles situados en esquinas opuestas de la imagen.
- **Sensibilidad al Desplazamiento:** Un MLP no reconoce un objeto si se traslada unos pocos píxeles en la imagen, ya que la entrada vectorizada cambia por completo.

8.1.1.2 1.2 Principios de Invariancia Espacial

Para procesar señales estructuradas en 2D de forma eficiente, las Redes Neuronales Convolucionales (CNN) incorporan dos sesgos inductivos clave:

1. **Localidad (*Local Receptive Fields*):** Las características visuales significativas (bordes, esquinas, texturas) se forman por la interacción de píxeles contiguos en regiones espaciales reducidas.

1. **Invariancia a la Traslación / Compartición de Pesos (*Weight Sharing*):** La representación o patrón de un objeto no debe cambiar independientemente de la posición que ocupe dentro de la imagen. Un detector de bordes es útil tanto en la esquina superior izquierda como en el centro.

8.1.2 2. Operaciones Convolucionales y sus Parámetros

8.1.2.1 2.1 La Operación de Convolución 2D

Un filtro o núcleo (*kernel*) $K \in \mathbb{R}^{k_h \times k_w}$ se desliza sobre la matriz de entrada $X \in \mathbb{R}^{H \times W}$, calculando el producto escalar elemento a elemento seguido de una suma:

$$Y_{i, j} = (X * K)_{i, j} = \{}sum_{m} \{}sum_{n} X_{i+m, j+n} \{}cdot K_{m, n}$$

```
Entrada X (5x5)           Kernel K (3x3)          Feature Map Y (3x3)

[ 1  1  1  0  0 ]        [ 1  0  1 ]

[ 0  1  1  1  0 ]   *    [ 0  1  0 ]     --->     [ 4  3  4 ]

[ 0  0  1  1  1 ]        [ 1  0  1 ]              [ .  .  . ]

[ 0  0  1  1  0 ]                                 [ .  .  . ]

[ 0  1  1  0  0 ]
```

8.1.2.2 2.2 Hiperparámetros Fundamentales: Padding, Stride y Canales

- **Relleno (*Padding*, p):** Añade filas y columnas de ceros alrededor del borde de la entrada para controlar la dimensión espacial de salida y evitar la pérdida acelerada de información en los bordes.
- **Paso (*Stride*, s):** Especifica el número de píxeles que se desplaza el filtro en cada paso horizontal y vertical. Un $s > 1$ subsamplea la entrada reduciendo su dimensión espacial.
- **Múltiples Canales de Entrada y Salida ($C_{\text{in}}, C_{\text{out}}$):**
- Si la entrada tiene C_{in} canales (ej. $RGB = 3$), el filtro posee dimensiones $C_{\text{in}} \times k_h \times k_w$.
- Para extraer C_{out} mapas de características distintos, se aplican C_{out} filtros independientes de este tipo.

8.1.2.3 2.3 Ecuación de Dimensión de Salida

Dada una entrada de dimensión $C_{\text{in}} \times H \times W$, un kernel de tamaño $k_h \times k_w$, padding p y stride s, la dimensión del mapa de características resultante $C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}$ se calcula mediante:

$$H_{\{}text{out}} = \{}left\{}lfloor \{}frac{H + 2p - k_h}{s} \{}right\{}rfloor + 1$$

$$W_{\{}text{out}} = \{}left\{}lfloor \{}frac{W + 2p - k_w}{s} \{}right\{}rfloor + 1$$

8.1.3 3. Capas de Agregación (*Pooling*)

Las capas de pooling introducen **invariancia local a pequeñas traslaciones y rotaciones**, reduciendo la dimensión espacial de las representaciones y disminuyendo la carga computacional.

8.1.3.1 3.1 Tipos Principales de Pooling

1. **Max Pooling (‘nn.MaxPool2d‘):** Extrae el valor máximo dentro de la ventana de selección. Resalta las características más prominentes o activaciones más fuertes.

1. **Average Pooling (‘nn.AvgPool2d‘):** Calcula el promedio de todos los valores de la ventana. Proporciona una representación más suave de la región.

Propiedad importante: Las capas de pooling operan de forma independiente sobre cada canal y **no poseen parámetros entrenables** (W, b).

8.1.4 4. Jerarquía de Representaciones y Arquitecturas Clásicas

8.1.4.1 4.1 Extracción Jerárquica de Características

A medida que la señal avanza a través de capas convolucionales profundas y subsampling:

- **Capas Iniciales:** Filtran patrones simples y locales de bajo nivel (bordes, gradientes de color, orientaciones).
- **Capas Intermedias:** Combinan los bordes para detectar texturas, esquinas y partes de objetos.
- **Capas Profundas:** Ensamblan las partes para representar objetos completos y estructuras semánticas complejas.

```
1
2 Imagen Raw    Bordes/Líneas    Texturas/Partes    Objetos Complejos    Clases (Logits)
3
4  (Entrada)      (Capa Conv 1)     (Capa Conv 2)       (Capa Conv 3)     (Capas Densas
     )
```

8.1.4.2 4.2 Arquitectura Clásica: LeNet-5 (Yann LeCun, 1998)

Fue una de las primeras redes convolucionales con éxito comercial para la clasificación de dígitos manuscritos (MNIST).

- **Estructura Típica:**

1. **Entrada:** Imagen en escala de grises ($1 \times 28 \times 28$ o $1 \times 32 \times 32$).
1. **Bloque 1:** `Conv2d` ($1 \to 6$ canales, 5×5) + `Tanh` + `AvgPool2d` (2×2, $s = 2$).
1. **Bloque 2:** `Conv2d` ($6 \to 16$ canales, 5×5) + `Tanh` + `AvgPool2d` (2×2, $s = 2$).
1. **Clasificador denso:** Aplanado (`Flatten`) + `Linear` (120) + `Tanh` + `Linear` (84) + `Tanh` + `Linear` (10 clases).

8.1.5 5. Gestión de Datos en PyTorch: `Dataset` y `DataLoader`

Para alimentar eficientemente datos e imágenes a modelos de Deep Learning, PyTorch estandariza la carga mediante dos clases fundamentales:

1. **'torch.utils.data.Dataset':** Almacena las muestras y sus etiquetas correspondientes. Debe implementar tres métodos:

- `__init__`: Inicializa rutas, transformaciones y metadatos.
- `__len__`: Devuelve el número total de muestras en el conjunto de datos.
- `__getitem__(self, idx)`: Carga y retorna una muestra (X, y) dado un índice idx.

1. **'torch.utils.data.DataLoader':** Envoltorio que abstrae el procesamiento iterativo en minilotes (*mini-batches*), barajado de datos (*shuffling*), carga paralela multiproceso (*multiprocessing*) y gestión de memoria.

8.1.6 6. Práctica y Aplicación en Código (PyTorch y Torchvision)

A continuación se incluye una implementación modular completa que incluye la definición de un `Dataset` personalizado, la canalización de transformaciones, la arquitectura CNN tipo LeNet-5 y el bucle de entrenamiento:

```

import torch

import torch.nn as nn

import torch.optim as optim

from torch.utils.data import Dataset, DataLoader

import torchvision.transforms as transforms

from PIL import Image

import os



# ------------------------------------------------------------------------

# 1. Pipeline de Transformaciones y Preprocesamiento

# ------------------------------------------------------------------------

IMAGE_SIZE = 32



transform_train = transforms.Compose([

    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),

    transforms.RandomHorizontalFlip(p=0.5),

    transforms.ToTensor(),

    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

])



```

```
transform_val_test = transforms.Compose([

    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),

    transforms.ToTensor(),

    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

])



# ------------------------------------------------------------------------

# 2. Definición de la Arquitectura CNN (Inspirada en LeNet-5)

# ------------------------------------------------------------------------

class LeNet5Classifier(nn.Module):

    def __init__(self, num_classes=10):

        super(LeNet5Classifier, self).__init__()



        # Bloque de Extracción de Características (Convolución + Pooling)

        self.feature_extractor = nn.Sequential(

            # Conv1: (B, 3, 32, 32) -> (B, 6, 28, 28)

            nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5, stride=1, padding=0),

            nn.Tanh(),

            # Pool1: (B, 6, 28, 28) -> (B, 6, 14, 14)

            nn.AvgPool2d(kernel_size=2, stride=2),



            # Conv2: (B, 6, 14, 14) -> (B, 16, 10, 10)

            nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5, stride=1, padding=0),

            nn.Tanh(),

            # Pool2: (B, 16, 10, 10) -> (B, 16, 5, 5)

            nn.AvgPool2d(kernel_size=2, stride=2)

        )



```

```
        # Bloque Clasificador Denso

        self.classifier = nn.Sequential(

            nn.Flatten(), # (B, 16 * 5 * 5) = (B, 400)

            nn.Linear(16 * 5 * 5, 120),

            nn.Tanh(),

            nn.Dropout(p=0.3),

            nn.Linear(120, 84),

            nn.Tanh(),

            nn.Linear(84, num_classes) # Logits de salida

        )


    def forward(self, x):

        x = self.feature_extractor(x)

        logits = self.classifier(x)

        return logits


# ----------------------------------------------------------------------

# 3. Bucle de Entrenamiento

# ----------------------------------------------------------------------

def train_cnn_demo():

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")


    # Modelo, Pérdida y Optimizador

    model = LeNet5Classifier(num_classes=10).to(device)

    criterion = nn.CrossEntropyLoss()

    optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)


    # Tensor sintético de prueba: 16 imágenes RGB de 32x32

    dummy_images = torch.randn(16, 3, 32, 32).to(device)

```

```
    dummy_labels = torch.randint(0, 10, (16,)).to(device)



    model.train()

    optimizer.zero_grad()



    # Forward Pass

    logits = model(dummy_images)

    loss = criterion(logits, dummy_labels)



    # Backward Pass

    loss.backward()

    optimizer.step()



    print(f"Demostración exitosa en {device}.")

    print(f"Dimensión de Salida (Logits): {logits.shape}")

    print(f"Pérdida Inicial (Loss): {loss.item():.4f}")



if __name__ == "__main__":

    train_cnn_demo()
```

Dime si así te permite copiar todo el contenido sin problemas y si te gustaría que aclaremos algún concepto o que preparemos un cuestionario interactivo de este tema para preparar tu evaluación.

8.2 Conexión con las prácticas

Las prácticas implementan convolución desde cero, kernels aprendidos, nn.Conv2d, pooling, LeNet y clasificación de imágenes; las figuras guardadas muestran las representaciones y métricas producidas.

8.2.1 Fuente práctica: `09_02_CNN.ipynb`

Ruta: code/09_02_CNN/09_02_CNN.ipynb

Apartados detectados: Short Assignment.

Salidas gráficas almacenadas: 5.

Salida textual conservada:

(500, 'cat.106.jpg') | (500, 'dog.10002.jpg') | (224, 300, 3) | (307, 410, 3)

Fragmento representativo:

```

import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
from matplotlib import pyplot as plt
import os #for managing files, folders, etc.
```

8.2.2 Fuente práctica: 09_04_CNN.ipynb

Ruta: code/09_02_CNN/09_04_CNN.ipynb

Apartados detectados: Image convolution from scratch; Learning kernels; nn.Conv2d; Pooling layers; MNIST Fashion with feature extraction.

Salidas gráficas almacenadas: 7.

Salida textual conservada:

<matplotlib.image.AxesImage at 0x7ebcbd8274d0> | <Figure size 640x480 with 1 Axes> | (396, 312, 3) | torch.Size([454, 370])

Fragmento representativo:

```

import torch
import torch.nn as nn
import torch.nn.functional as f
from matplotlib import pyplot as plt
from torchvision import transforms
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
```

8.2.3 Fuente práctica: 09_06_CNN.ipynb

Ruta: code/09_02_CNN/09_06_CNN.ipynb

Apartados detectados: Stacking two convolutional layers; Classical LeNet; Modern LeNet; Dogs vs. Cats Classification with a Custom CNN.

Salidas gráficas almacenadas: 1.

Salida textual conservada:

Epoch [1/5] | Train Loss: 0.5030 | Train Acc: 81.61% | Test Acc: 86.03% | Epoch [2/5] | Train Loss: 0.3258 | Train Acc: 88.17% | Test Acc: 88.36% | Epoch [3/5] | Train Loss: 0.2807 | Train Acc: 89.67% | Test Acc: 89.13% | Epoch [4/5] | Train Loss: 0.2526 | Train Acc: 90.75% | Test Acc: 89.76%

Fragmento representativo:

```

import torch
import torch.nn as nn
import torch.nn.functional as f
from matplotlib import pyplot as plt
import torchvision
from torchvision import transforms
from torch.utils.data import Dataset, DataLoader
```

8.3 Autoevaluación razonada

Pregunta

¿Qué controlan padding y stride?

Solución. Padding controla el borde y el tamaño espacial disponible; stride controla cuánto se desplaza el kernel y, por tanto, la reducción espacial.

Pregunta

¿Por qué se comparten pesos en una CNN?

Solución. Para detectar el mismo patrón en distintas posiciones con muchos menos parámetros que una capa totalmente conectada.

8.4 Fuentes y trazabilidad

Diapositivas:

- `slides/09\_01\_CNN.pdf`
- `slides/09\_03\_CNN.pdf`
- `slides/09\_05\_CNN.pdf`

Nota personal: apuntes/Apuntes del Tema 9 - Redes Neuronales Convolucionales (CNN I y II).md

9 Datos secuenciales y redes recurrentes

9.1 Apuntes del Tema 10: Redes Neuronales Recurrentes

9.1.1 1. De Datos Estáticos a Datos Secuenciales

9.1.1.1 1.1 Limitaciones de los Modelos Feedforward (MLPs y CNNs)

- **Asunción de Muestras Independientes (*i.i.d.*):** Los modelos tradicionales como los Perceptrones Multicapa (MLP) o las Redes Convolucionales (CNN) asumen que las muestras de entrada son independientes e idénticamente distribuidas.
- **Entradas de Tamaño Fijo:** Un MLP o una CNN requieren un vector o tensor de dimensiones de entrada rígidamente predefinidas.
- **Incapacidad para Capturar Dependencias Temporales:** En datos con orden temporal o secuencial (como texto, audio, series temporales financieras o trayectorias GPS), la información del elemento actual depende de los elementos anteriores.

9.1.1.2 1.2 Modelos Autorregresivos y el Estado Oculto

Para modelar secuencias $X = (x_1, x_2, \ldots, x_T)$, deseamos estimar la probabilidad conjunta mediante la regla de la cadena de probabilidades:

$$P(x_1, x_2, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, \ldots, x_{t-1})$$

- **Modelos de Marcov (Orden k):** Asumen que x_t solo depende de los k pasos anteriores.
- **Mecanismo de Estado Oculto (*Hidden State*): En lugar de truncar la historia a k pasos, una Red Neuronal Recurrente (RNN) mantiene un** estado latente h_t** que actúa como una memoria comprimida de toda la secuencia procesada desde $t = 1$ hasta $t - 1$.

9.1.2 2. Arquitectura de la Red Neuronal Recurrente Básica (RNN Vanilla)

9.1.2.1 2.1 Ecuaciones del Estado Oculto y Salida

En cada paso de tiempo t, la RNN recibe la entrada actual $x_t \in \mathbb{R}^d$ y el estado oculto del paso anterior $h_{t-1} \in \mathbb{R}^h$.

1. **Actualización del Estado Oculto:**

$$h_t = \{}phi\{}left(x_t W_{xh} + h_{t-1} W_{hh} + b_h\{}right)$$

1. **Cálculo de la Salida:**

$$\{}hat{y}_t = \{}text{Softmax}\{}left(h_t W_{hq} + b_q\{}right) \{}quad \{}text{o} \{}quad o_t = h_t W_{hq} + b_q$$

donde:

- $W_{xh} \in \mathbb{R}^{d \times h}$: Matriz de pesos para la entrada.
- $W_{hh} \in \mathbb{R}^{h \times h}$: Matriz de pesos para la transición recurrente de estados.
- $W_{hq} \in \mathbb{R}^{h \times q}$: Matriz de pesos para la capa de salida.
- $\phi(\cdot)$: Función de activación no lineal (típicamente tanh o ReLU).

Compartición de Pesos (*Weight Sharing* Temporal): Las matrices de pesos $\{W_{xh}, W_{hh}, W_{hq}\}$ son exactamente las mismas en todos los pasos de tiempo $t = 1, \ldots, T$.

```
1
2  Estructura Desplegada en el Tiempo (Unfolded RNN):
3
4
5
6    y_1          y_2                  y_T
7
8
9
10
11
12
13
14   h_1       h_2      ...      h_T
15
16       W_hh W_hh
17
18
19
20       W_xh         W_xh                 W_xh
21
22    x_1          x_2                  x_T
```

9.1.3 3. Entrenamiento: Retropropagación a través del Tiempo (BPTT)

9.1.3.1 3.1 Backpropagation Through Time (BPTT)

Para entrenar las RNNs, la red recurrente se "despliega" a lo largo de los T pasos temporales formando un grafo acíclico dirigido. La pérdida total es la suma de las pérdidas en cada paso de tiempo:

$$\{}mathcal{L}(W) = \{}sum_{t=1}^{T} \{}mathcal{L}_t(y_t, \{}hat{y}_t)$$

Para calcular el gradiente respecto a la matriz de transición recurrente W_{hh}, se aplica la regla de la cadena acumulando las dependencias pasadas:

$$\{}frac{\{}partial \{}mathcal{L}_t}{\{}partial W_{hh}} = \{}sum_{k=1}^{t} \{}frac{\{}partial \{}mathcal{L}_t}{\{}partial h_t} \{}cdot \{}left(\{}prod_{j=k+1}^{t} \{}frac{\{}partial h_j}{\{}partial h_{j-1}} \{}right) \{}cdot \{}frac{\{}partial h_k}{\{}partial W_{hh}}$$

9.1.3.2 3.2 Problema Crítico: Desvanecimiento y Explosión del Gradiente

El término $\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}}$ implica multiplicar repetidamente la matriz de pesos $W_{hh}^{\top}$.

- **Desvanecimiento del Gradiente (*Vanishing Gradients*): **Si los autovalores de W_{hh} son menores a 1, la magnitud del gradiente decrece exponencialmente hacia cero cuando la secuencia es larga ($T \gg 0$). La red** olvida las dependencias a largo plazo**.
- **Explosión del Gradiente (*Exploding Gradients*): **Si los autovalores son mayores a 1, el gradiente crece exponencialmente destruyendo el entrenamiento. Se soluciona mediante** Gradient Clipping**:

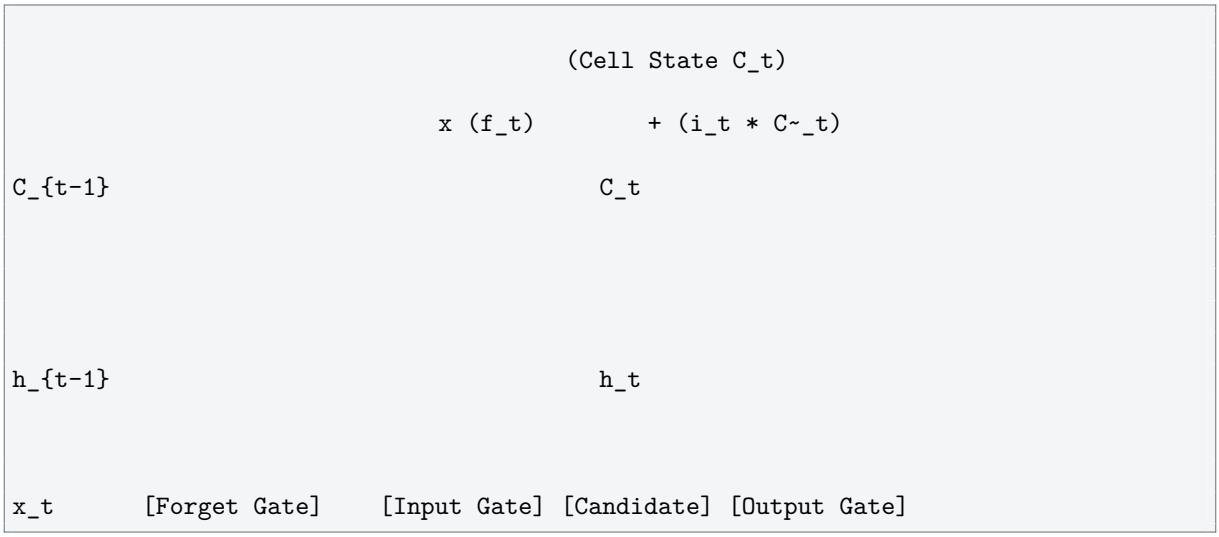
$$g \{}leftarrow \{}min\{}left(1, \{}frac{\{}theta}{\{}Vert{}g\{}Vert{}}\{}right) g$$

9.1.4 4. Redes Gated: LSTM y GRU

Para solucionar el desvanecimiento de gradiente y permitir recordar información a largo plazo, se crearon arquitecturas basadas en puertas de control (*gates*).

9.1.4.1 4.1 LSTM (*Long Short-Term Memory*)

La LSTM sustituye la celda estándar por una estructura de memoria que mantiene un **estado de celda** C_t (*Cell State* o autopista de información) y un **estado oculto** h_t.



9.1.4.2 Puertas y Ecuaciones Matemáticas de la LSTM:

1. **Forget Gate (f_t):** Decide qué información descartar del estado de celda anterior.

$$f_t = \sigma\left(x_t W_{xf} + h_{t-1} W_{hf} + b_f\right)$$

1. **Input Gate (i_t) y Candidato ($\tilde{C}_t$):** Deciden qué nueva información almacenar.

$$i_t = \sigma\left(x_t W_{xi} + h_{t-1} W_{hi} + b_i\right)$$
$$\tilde{C}_t = \tanh\left(x_t W_{xc} + h_{t-1} W_{hc} + b_c\right)$$

1. **Actualización del Cell State (C_t):**

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

1. **Output Gate (o_t) y Estado Oculto (h_t):**

$$o_t = \sigma\left(x_t W_{xo} + h_{t-1} W_{ho} + b_o\right)$$
$$h_t = o_t \odot \tanh(C_t)$$

9.1.4.3 4.2 GRU (*Gated Recurrent Unit*)

Variante simplificada de la LSTM que combina el estado de celda y el estado oculto en una sola variable h_t, reduciendo el coste computacional y el número de parámetros.

1. **Reset Gate (r_t):** Determina cuánta información del pasado olvidar:

$$r_t = \sigma\left(x_t W_{xr} + h_{t-1} W_{hr} + b_r\right)$$

1. **Update Gate (z_t):** Actúa simultáneamente como puerta de olvida e entrada:

$$z_t = \sigma\left(x_t W_{xz} + h_{t-1} W_{hz} + b_z\right)$$

1. **Candidato de Estado Oculto ($\tilde{h}_t$):**

$$\tilde{h}_t = \tanh\left(x_t W_{xh} + (r_t \odot h_{t-1}) W_{hh} + b_h\right)$$

1. **Estado Oculto Final (h_t):**

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

9.1.5 5. RNNs Bidireccionales y Apiladas (*Deep RNNs*)

- **RNN Bidireccional (BiRNN / BiLSTM):** Procesa la secuencia en ambas direcciones simultáneamente usando dos estados ocultos independientes ($\overrightarrow{h}_t$ hacia adelante y $\overleftarrow{h}_t$ hacia atrás). El estado combinado es $h_t = [\overrightarrow{h}_t ; \overleftarrow{h}_t]$.

- *Uso:* Crucial en tareas de procesamiento de lenguaje natural (NLP) donde el contexto futuro influye en el significado del presente.

- **RNN Profunda / Apilada (*Stacked RNN*):** Apila múltiples capas recurrentes donde la salida del estado oculto de la capa l sirve como entrada x_t para la capa $l+1$.

9.1.6 6. Práctica y Aplicación en Código (PyTorch)

A continuación se incluye una implementación modular completa que ilustra la construcción de una red **LSTM Bidireccional** para la clasificación de secuencias (ej. análisis de sentimiento o series temporales) en PyTorch:

```
1
2  import torch
3
4  import torch.nn as nn
5
6  import torch.optim as optim
7
8
9
10 # ------------------------------------------------------------------------
11
12 # 1. Definición del Modelo BiLSTM / GRU en PyTorch
13
14 # ------------------------------------------------------------------------
15
16 class SequenceClassifier(nn.Module):
17
18     def __init__(self, vocab_size=1000, embed_dim=64, hidden_dim=128, num_classes=2,
       num_layers=2, dropout=0.3):
19
20         super(SequenceClassifier, self).__init__()
21
22
23
24         # Capa de Embedding para convertir índices de tokens en vectores continuos
25
26         self.embedding = nn.Embedding(vocab_size, embed_dim)
27
28
29
30         # Capa LSTM Bidireccional de múltiples capas
31
32         self.lstm = nn.LSTM(
33
34             input_size=embed_dim,
35
36             hidden_size=hidden_dim,
37
38             num_layers=num_layers,
39
40             batch_first=True,       # Entrada esperada: (Batch, Seq_Len, Embed_Dim)
41
42             bidirectional=True,     # BiLSTM
43
44             dropout=dropout if num_layers > 1 else 0.0
45
46         )
47
48
49
50         # Capa lineal final: hidden_dim * 2 por la bidireccionalidad
51
```

```
        self.fc = nn.Linear(hidden_dim * 2, num_classes)

        self.dropout = nn.Dropout(p=dropout)


    def forward(self, x):

        # x shape: (Batch, Seq_Len)

        embedded = self.embedding(x) # (Batch, Seq_Len, Embed_Dim)


        # output shape: (Batch, Seq_Len, Hidden_Dim * 2)

        # h_n, c_n shape: (Num_Layers * 2, Batch, Hidden_Dim)

        lstm_out, (h_n, c_n) = self.lstm(embedded)


        # Concatenar el último estado oculto forward y backward de la última capa

        hidden_forward = h_n[-2, :, :]

        hidden_backward = h_n[-1, :, :]

        last_hidden = torch.cat((hidden_forward, hidden_backward), dim=1) # (Batch, Hidden_Dim * 2)


        out = self.dropout(last_hidden)

        logits = self.fc(out) # (Batch, Num_Classes)

        return logits


# ------------------------------------------------------------------------

# 2. Ejemplo de Bucle de Entrenamiento

# ------------------------------------------------------------------------

def train_rnn_demo():

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")


    # Hiperparámetros

    VOCAB_SIZE = 1000

    BATCH_SIZE = 16
```

```
109
110     SEQ_LEN = 32
111
112     NUM_CLASSES = 2
113
114
115
116     # Modelo, pérdida y optimizador
117
118     model = SequenceClassifier(
119
120         vocab_size=VOCAB_SIZE,
121
122         embed_dim=64,
123
124         hidden_dim=128,
125
126         num_classes=NUM_CLASSES,
127
128         num_layers=2
129
130     ).to(device)
131
132
133
134     criterion = nn.CrossEntropyLoss()
135
136     optimizer = optim.Adam(model.parameters(), lr=0.001)
137
138
139
140     # Datos sintéticos de secuencia: (Batch, Seq_Len)
141
142     dummy_input = torch.randint(0, VOCAB_SIZE, (BATCH_SIZE, SEQ_LEN)).to(device)
143
144     dummy_labels = torch.randint(0, NUM_CLASSES, (BATCH_SIZE,)).to(device)
145
146
147
148     model.train()
149
150     optimizer.zero_grad()
151
152
153
154     # Forward Pass
155
156     logits = model(dummy_input)
157
158     loss = criterion(logits, dummy_labels)
159
160
161
162     # Backward Pass
163
164     loss.backward()
165
166
```

```
    # Gradient Clipping esencial para evitar la explosión de gradiente en RNNs
    nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)


    optimizer.step()


    print(f"Demostración RNN completada en dispositivo: {device}")
    print(f"Dimensión de salida (Logits): {logits.shape}")
    print(f"Pérdida inicial (Loss): {loss.item():.4f}")


if __name__ == "__main__":
    train_rnn_demo()
```

9.2 Conexión con las prácticas

Los notebooks pasan del modelado de secuencias con MLP a una RNN implementada desde cero para predicción del siguiente carácter y exponen la necesidad de controlar el gradiente.

9.2.1 Fuente práctica: 10_01_SequentialData.ipynb

Ruta: code/10_01_SequentialData/10_01_SequentialData.ipynb

Apartados detectados: Character-level Language Modeling with MLPs.

Salidas gráficas almacenadas: 0.

Salida textual conservada:

> 'malena es un cuy' | — Starting training: OneHotMLP — | Epoch [1/10] - Avg Loss: 2.6257 | Epoch [2/10] - Avg Loss: 2.2111

Fragmento representativo:

```

import os
import math
from collections import Counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
14
15 import torch.optim as optim
16 from torch.utils.data import Dataset
17 from torch.utils.data import DataLoader
```

9.2.2 Fuente práctica: 10_02_RNNfromScratch.ipynb

Ruta: code/10_02_RNN/10_02_RNNfromScratch.ipynb

Apartados detectados: Implementing a Recurrent Neural Network from scratch for next-character prediction.

Salidas gráficas almacenadas: 0.

Salida textual conservada:

> Device: cuda | Number of characters: 24937 | Walt Whitman has somewhere a fine and just distinction between “loving | by allowance” and “loving with personal love.” This distinction applies

Fragmento representativo:

```
1
2 import torch
3 import torch.nn as nn
4 from torch.utils.data import Dataset, DataLoader
5 import math
6 import random
```

9.3 Autoevaluación razonada

Pregunta

¿Qué información conserva el estado oculto?

Solución. Una representación del contexto procesado hasta el paso temporal actual.

Pregunta

¿Qué es gradient clipping?

Solución. Una limitación de la norma del gradiente para evitar actualizaciones excesivas, especialmente útil frente a explosión del gradiente.

9.4 Fuentes y trazabilidad

Diapositivas:

- slides/10_01_SequentialData.pdf
- slides/10_02_RNN.pdf

Nota personal: apuntes/Apuntes del Tema 10 - Redes Neuronales Recurrentes.md

10 Atención y Transformers

10.1 Apuntes del Tema 11: Attention Mechanisms and Transformers

10.1.1 1. Introducción y Limitaciones de los Modelos Secuenciales (Seq2Seq)

10.1.1.1 1.1 El Enfoque Seq2Seq Tradicional (RNNs)

- **Modelado Secuencial:** Las redes neuronales recurrentes (RNNs) procesan las secuencias token a token de forma secuencial.
- **Arquitectura Encoder-Decoder:** En tareas como la traducción automática (p. ej., de inglés a español), el *Encoder* procesa la frase de entrada y condensa su información en un vector de estado oculto final h_{T_x}. Luego, el *Decoder* genera la secuencia traducida token a token condicionado en este vector.

```
1
2 Entrada (Inglés)    [ ENCODER (RNN) ]    Vector de Estado H_Tx    [ DECODER (RNN) ]
    Salida (Español)
```

10.1.1.2 1.2 El Cuello de Botella del Vector de Contexto (*Context Bottleneck*)

- **Limitación Principal:** Comprimir toda una frase de longitud variable en un único vector de tamaño fijo h_{T_x} genera una gran pérdida de información.
- **Fallo en Secuencias Largas:** Aunque funciona bien para frases cortas (p. ej., *"The black cat"* $\rightarrow$ *"El gato negro"*), el modelo falla en oraciones largas y complejas.
- **Dependencias a Largo Plazo:** Las palabras iniciales se "olvidan" a medida que la secuencia crece. Además, no gestiona eficientemente el reordenamiento de palabras entre idiomas.

10.1.2 2. Mecanismos de Atención en RNNs (Bahdanau et al., 2014)

10.1.2.1 2.1 Filosofía de la Atención

En lugar de depender de un único vector fijo, la atención permite al *Decoder* acceder a **todos** los estados ocultos del *Encoder* $\{h_1, h_2, \ldots, h_{T_x}\}$ en cada paso de decodificación.

- **Adaptabilidad:** En cada paso t, el *Decoder* calcula qué partes de la frase de entrada son más relevantes para predecir el siguiente token.

- **Vector de Contexto Dinámico (c_t):** Se calcula como una suma ponderada de todos los estados del *Encoder*:

$$c_t = \{}sum_{i=1}^{T_x} \{}alpha_{t,i} h_i$$

10.1.2.2 2.2 Formulación Matemática de la Atención de Bahdanau

Para cada paso de tiempo del decodificador i con estado anterior s_{i-1}:

1. **Puntuación de Alineamiento (*Score*):**

$$e_{ij} = a(s_{i-1}, h_j)$$
(donde a es una red feedforward).

1. **Pesos de Atención (Softmax):**

$$\{}alpha_{ij} = \{}frac{\{}exp(e_{ij})}{\{}sum_{k=1}^{T_x} \{}exp(e_{ik})}$$

1. **Vector de Contexto:**

$$c_i = \{}sum_{j=1}^{T_x} \{}alpha_{ij} h_j$$

1. **Actualización del Estado y Predicción:**

$$s_i = f(s_{i-1}, y_{i-1}, c_i)$$
$$P(y_i \{}mid y_{1:i-1}, x) = g(y_{i-1}, s_i, c_i)$$

10.1.2.3 2.3 Inconvenientes de la Atención sobre RNNs

- **Dependencia Recurrente:** La computación sigue siendo secuencial (paso a paso), lo que impide la paralelizabilidad masiva en GPUs durante el entrenamiento.
- **Poco Directo:** La atención solo conecta el *Decoder* con el *Encoder*, pero la información dentro de cada bloque sigue fluyendo de forma secuencial implícita.

10.1.3 3. Autoatención (*Self-Attention*) en Transformers

Propuesto por Vaswani et al. (2017) en *"Attention is All You Need"*, elimina por completo la recurrencia. Cada palabra puede atender directamente a cualquier otra palabra de la secuencia sin importar la distancia.

10.1.3.1 3.1 Queries, Keys y Values (Q, K, V)

Para cada token con embedding $x \in \mathbb{R}^d$, se calculan tres proyecciones mediante matrices de pesos entrenables:

1. **Query ($q = W_Q x \in \mathbb{R}^{d_k}$):** Lo que el token busca saber sobre otros.
1. **Key ($k = W_K x \in \mathbb{R}^{d_k}$):** Cómo se identifica o etiqueta el token ante los demás.
1. **Value ($v = W_V x \in \mathbb{R}^{d_v}$):** La información que el token lleva para compartir.

10.1.3.2 3.2 Formulación Matricial: Scaled Dot-Product Attention

Dada la matriz de entrada $X \in \mathbb{R}^{T_x \times d_{\text{model}}}$, se proyectan las matrices $Q = XW_Q$, $K = XW_K$ y $V = XW_V$.

$$\{}text{Attention}(Q, K, V) = \{}text{Softmax}\{}left(\{}frac{Q K^\{}top}{\{}sqrt{d_k}} \{}right) V$$

- **Matriz de Similitud ($S = \frac{QK^\top}{\sqrt{d_k}}$):** Mide la afinidad entre las consultas y las claves.
- **Factor de Escala ($\sqrt{d_k}$):** Previene que el producto escalar alcance magnitudes muy grandes que saturarían la función Softmax (provocando gradientes nulos).

```
1
2  Entrada X (T x d_model)
3
4        Q = X·W_Q (T x d_k)
5
6        K = X·W_K (T x d_k)      [ Q·K / √d_k ]    [ Softmax ]
7
8        V = X·W_V (T x d_v)                           Salida Z (T x d_v)
```

10.1.4 4. Atención Multicabeza (*Multi-Head Self-Attention*)

10.1.4.1 4.1 Motivación

Una sola cabeza de atención solo proyecta en un espacio de subespacio único y puede capturar un único tipo de relación (p. ej., sujeto-verbo). El lenguaje requiere capturar múltiples relaciones simultáneamente (sintácticas, semánticas, modificadores).

10.1.4.2 4.2 Ecuaciones de Multi-Head Attention

Se entrenan h conjuntos independientes de matrices de proyección $\{W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}\}$:

$$\{}text{head}_i = \{}text{Attention}\{}left(Q^\{(i)}, K^\{(i)}, V^\{(i)}\{}right) = \{}text{Attention}\{}l
W_Q^\{(i)}, \{}, X W_K^\{(i)}, \{}, X W_V^\{(i)}\{}right)$$

Las salidas de todas las cabezas se concatenan y se proyectan linealmente con $W_O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ para mantener las dimensiones de entrada:

$$\{}text{MultiHead}(X) = \{}text{Concat}(\{}text{head}_1, \{}text{head}_2, \{}dots, \{}text{head}_h) W_O$$

10.1.4.3 4.3 Dimensiones de Tensores en Práctica (Batching)

- Entrada $X \in \mathbb{R}^{B \times T \times d_{\text{model}}}$ (B: Batch size, T: Longitud de secuencia).
- Proyección y cambio de forma para procesamiento paralelo:

$$Q, K, V \{}rightarrow \{}mathbb{R}^{B \{}times h \{}times T \{}times d_k}$$

- Salida de la atención por cabeza: $\mathbb{R}^{B \times h \times T \times d_v}$.
- Concatenación y proyección final: $\mathbb{R}^{B \times T \times d_{\text{model}}}$.

10.1.5 5. Arquitectura del Transformer: Encoder

El *Encoder* mapea la secuencia de entrada en una representación contextualizada manteniendo la dimensión del eje temporal T y la dimensión del embedding d_{model}. Consta de un apilamiento de N capas idénticas ($N = 6$ en la arquitectura original).

```
Entrada (Tokens)    [ Input Embedding ]    (+)    [ Stack de N Capas Encoder ]
    Representación Contextualizada



                                    [ Positional Encoding ]
```

10.1.5.1 5.1 Codificación Posicional (*Positional Encoding*)

Dado que la autoatención es invariante a permutaciones (trata las palabras como un conjunto no ordenado), se debe añadir explícitamente información de orden.

- **Incorporación:** Se suma directamente al embedding de entrada en el primer nivel: $z_i = x_i + p_{\text{pos}}$.
- **Sinusoidal Encoding (Original Transformer):**

$$PE_{(\{}text{pos}, 2i)} = \{}sin\{}left(\{}frac{\{}text{pos}}{10000^{2i / d_{\{}text{model}}}}\{}righ
$$PE_{(\{}text{pos}, 2i+1)} = \{}cos\{}left(\{}frac{\{}text{pos}}{10000^{2i / d_{\{}text{model}}}}\{}

- **Learned Positional Embeddings (BERT, GPT):** Tabla de parámetros entrenables $P \in \mathbb{R}^{T_{\max} \times d_{\text{model}}}$.

10.1.5.2 5.2 Conexiones Residuales y Normalización por Capa (Add & Norm)

Se aplican alrededor de cada subcapa (atención y red feedforward):

$$\{}text{Output} = \{}text{LayerNorm}(x + \{}text{Sublayer}(x))$$

- **Residual Connections:** Añaden la entrada de la capa a la salida ($x + \text{Sublayer}(x)$), facilitando el flujo del gradiente sin desvanecimiento ni explosión.
- **Layer Normalization:** Normaliza a través de las características d_{model} independientemente para cada token.
- **Variantes de Orden:**
- *Post-LN (Original):* $\text{LN}(x + \text{Sublayer}(x))$.
- *Pre-LN (Moderno - GPT/BERT):* $x + \text{Sublayer}(\text{LN}(x))$, mejora la estabilidad en redes muy profundas.

10.1.5.3 5.3 Red Feedforward (FFN)

Se aplica de forma independiente a cada token (*position-wise*):

$$\{}text{FFN}(x) = \{}text{Activation}(x W_1 + b_1) W_2 + b_2$$

- Expande la dimensión intermedia a d_{ff} (típicamente $d_{ff} = 4 \times d_{\text{model}}$).
- **Funciones de Activación:**
- *ReLU:* $\max(0, x)$ (Transformer original).
- *GELU:* $x \cdot \Phi(x)$ (Modelos modernos como BERT o GPT).

10.1.6 6. Arquitectura del Transformer: Decoder

El *Decoder* genera la secuencia objetivo token a token en modo autorregresivo.

```

Entrada desplazada    [ Output Embedding + PE ]    [ Masked Self-Attention ]    [
    Cross-Attention ]    [ FFN ]    [ Linear + Softmax ]
```

10.1.6.1 6.1 Masked Multi-Head Self-Attention (Atención Causal)

Para evitar que el modelo acceda a tokens futuros durante el entrenamiento, se aplica una **máscara causal** sobre la matriz de puntuaciones previa al Softmax:

$$\{}text{scores}_{\{}text{masked}} = \{}text{scores} + M, \{}quad \{}text{donde } M_{ij} = \{}begin{cases} 0 & \{}text{si } i \{}ge j \{}\{} -\{}infty & \{}text{si } i < j \{}end{cases}$$

Esto asegura que en la posición t, la predicción solo dependa de los tokens anteriores $y_{<t}$.

10.1.6.2 6.2 Cross-Attention (Atención Cruzada Encoder-Decoder)

Conecta la representación del *Encoder* con el *Decoder*:

- **Query (Q):** Proviene del *Decoder* ($X_{\text{dec}} W_Q$).
- **Key (K) y Value (V):** Provienen de la salida final del *Encoder* ($H_{\text{enc}} W_K$, $H_{\text{enc}} W_V$).

$$\{}text{CrossAttention}(Q, K, V) = \{}text{Softmax}\{}left(\{}frac{Q K^\{}top}{\{}sqrt{d_k}} \{}right) V$$

10.1.6.3 6.3 Capas Finales y Proyección de Salida

1. La salida oculta del decodificador $H_{\text{dec}} \in \mathbb{R}^{B \times T_y \times d_{\text{model}}}$ pasa por una capa lineal que proyecta al tamaño del vocabulario V:

$$\{}text{logits} = H_{\{}text{dec}} W_{\{}text{vocab}}^\{}top + b$$

1. Se aplica **Softmax** para obtener la distribución de probabilidad $P(y_t \mid y_{<t}, x)$.

10.1.7 7. Consejos de Entrenamiento, Inferencia y Arquitecturas

10.1.7.1 7.1 Estrategias de Entrenamiento

- **Subword Tokenization:** Uso de algoritmos como BPE (Byte-Pair Encoding) o WordPiece para evitar la explosión de vocabulario y manejar palabras fuera del diccionario (OOV).

- **Padding Masking:** Dado que los batches se rellenan con `<PAD>` para igualar longitudes, se aplica un enmascaramiento con $-\infty$ para evitar atender a posiciones de relleno.
- **Warm-up Scheduling:** Incremento gradual de la tasa de aprendizaje durante los primeros pasos seguido de un decaimiento (p. ej., coseno).

10.1.7.2 7.2 Estrategias de Decodificación en Inferencia

- **Greedy Decoding:** Selecciona $y_t = \arg\max_v P(v)$. Rápido, pero puede caer en subóptimos locales.
- **Beam Search:** Mantiene las k mejores hipótesis parciales acumulando sus probabilidades logarítmicas.
- **Temperature Scaling:** Ajusta los logits dividiéndolos por T antes del Softmax para controlar la creatividad/aleatoriedad:
- $T < 1$: Distribución más picuda (más determinista).
- $T > 1$: Distribución más plana (más diversa).

10.1.7.3 7.3 Evolución de las Arquitecturas Transformer

- **Encoder-Only (p. ej., BERT):** Utiliza atención bidireccional. Ideal para tareas de comprensión, clasificación de textos y extracción de entidades.
- **Decoder-Only (p. ej., GPT-3, LLaMA, Mistral):** Utiliza atención causal. Orientado a la generación de texto autorregresiva.
- **Encoder-Decoder (p. ej., T5, BART):** Conserva la arquitectura original seq2seq para traducción y resumen.

10.1.8 8. Práctica en Código (PyTorch)

A continuación se incluye una implementación ejecutable de un módulo de **Multi-Head Self-Attention** nativo con enmascaramiento causal opcional:

```
1
2  import torch
3
4  import torch.nn as nn
5
6  import torch.nn.functional as F
7
8  import math
9
10
11
12 class MultiHeadSelfAttention(nn.Module):
13
14     def __init__(self, d_model=512, num_heads=8):
15
16         super(MultiHeadSelfAttention, self).__init__()
```

```
17
18          assert d_model % num_heads == 0, "d_model debe ser divisible por num_heads"
19
20
21
22          self.d_model = d_model
23
24          self.num_heads = num_heads
25
26          self.d_k = d_model // num_heads
27
28
29
30          # Proyecciones lineales para Q, K, V
31
32          self.W_q = nn.Linear(d_model, d_model, bias=False)
33
34          self.W_k = nn.Linear(d_model, d_model, bias=False)
35
36          self.W_v = nn.Linear(d_model, d_model, bias=False)
37
38
39
40          # Proyección de salida
41
42          self.W_o = nn.Linear(d_model, d_model, bias=False)
43
44
45
46      def forward(self, x, mask=None):
47
48          batch_size, seq_len, _ = x.size()
49
50
51
52          # 1. Proyección lineal -> (B, T, h, d_k) -> Transposición a (B, h, T, d_k)
53
54          Q = self.W_q(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose
    (1, 2)
55
56          K = self.W_k(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose
    (1, 2)
57
58          V = self.W_v(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose
    (1, 2)
59
60
61
62          # 2. Scaled Dot-Product Attention
63
64          scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)
65
66
67
68          if mask is not None:
69
70              scores = scores.masked_fill(mask == 0, -1e9)
71
```

```
72
73
74          attn_weights = F.softmax(scores, dim=-1)
75
76          attn_output = torch.matmul(attn_weights, V) # (B, h, T, d_k)
77
78
79
80          # 3. Concatenación de cabezas y proyección de salida
81
82          attn_output = attn_output.transpose(1, 2).contiguous().view(batch_size,
        seq_len, self.d_model)
83
84          output = self.W_o(attn_output)
85
86
87
88          return output
89
90
91
92  # Ejemplo de prueba rápida:
93
94  if __name__ == "__main__":
95
96      B, T, d_model = 2, 5, 64
97
98      x = torch.randn(B, T, d_model)
99
100
101
102     # Máscara Causal (Triangular Inferior)
103
104     causal_mask = torch.tril(torch.ones(T, T)).unsqueeze(0).unsqueeze(0)
105
106
107
108     mha = MultiHeadSelfAttention(d_model=d_model, num_heads=4)
109
110     out = mha(x, mask=causal_mask)
111
112     print(f"Dimensión del tensor de entrada: {x.shape}")
113
114     print(f"Dimensión del tensor procesado: {out.shape}")
```

10.2 Conexión con las prácticas

Las prácticas implementan atención, multi-head attention, codificaciones posicionales, Add-Norm, FFN y un encoder Transformer; los tensores de prueba y las máscaras muestran el flujo de dimensiones.

10.2.1 Fuente práctica: `11_01_Attention.ipynb`

Ruta: code/11_01_Attention/11_01_Attention.ipynb

Apartados detectados: 11_01 Attention mechanisms.

Salidas gráficas almacenadas: 0.

Fragmento representativo:

```

import math
import torch
import torch.nn as nn


class SelfAttention(nn.Module):

    def __init__(self, d_model, d_k, d_v):
        super(SelfAttention, self).__init__()

        self.W_Q = nn.Linear(d_model, d_k, bias =False)
        self.W_K = nn.Linear(d_model, d_k, bias =False)
        self.W_V = nn.Linear(d_model, d_v, bias =False)

        self.scale = math.sqrt(d_k)

    def forward(self, X):
        Q = self.W_Q(X)
        K = self.W_K(X)
        V = self.W_V(X)

        scores = torch.matmul(Q, K.transpose(-2, -1)) / self.scale

        attention_weights = torch.softmax(scores, dim=-1)

        outputs = torch.matmul(attention_weights, V)

        return outputs
```

10.2.2 Fuente práctica: 11_02_Attention.ipynb

Ruta: code/11_02_Attention/11_02_Attention.ipynb

Apartados detectados: 11_02 Multi-head attention and Transformer encoder; 1. The ‘MultiHeadSelfAttention‘ class; 2. The ‘SinusoidalPositionalEncoding‘ class; 3. The ‘LearnedPositionalEncoding‘ class; 4. The ‘AddNorm‘ class.

Salidas gráficas almacenadas: 2.

Salida textual conservada:

> Forma de la entrada: torch.Size([32, 10, 512]) | Forma de la salida : torch.Size([32, 10, 512]) | Input shape : torch.Size([2, 10, 64]) | Output shape: torch.Size([2, 10, 64])

Fragmento representativo:

```

import math
import torch
import torch.nn as nn
```

```
import matplotlib.pyplot as plt



class MultiHeadSelfAttention(nn.Module):

    def __init__(self, d_model, d_k, d_v, num_heads):
        super(MultiHeadSelfAttention, self).__init__()

        self.num_heads = num_heads
        self.d_k = d_k
        self.d_v = d_v
        self.d_model = d_model

        self.W_Q = nn.Linear(d_model, num_heads * d_k, bias=False)
        self.W_K = nn.Linear(d_model, num_heads * d_k, bias=False)
        self.W_V = nn.Linear(d_model, num_heads * d_v, bias=False)

        self.W_o = nn.Linear(num_heads * d_v, d_model, bias=False)

        self.scale = math.sqrt(d_k)

    def forward(self, x):
        B, T, _ = x.shape
```

10.2.3 Fuente práctica: 11_03_Attention.ipynb

Ruta: code/11_03_Attention/11_03_Attention.ipynb

Apartados detectados: 11_03 Transformer decoder; 1. 'MultiHeadAttention'; 2. 'DecoderLayer'; 3. Mask helper functions (additive masks); 4. Full 'Decoder' stack (batched).

Salidas gráficas almacenadas: 0.

Salida textual conservada:

> Cross-Attention Output Shape: torch.Size([2, 10, 64]) | Self-Attention Output Shape : torch.Size([2, 10, 64]) | Decoder Output shape: torch.Size([2, 10, 64]) | Logits Output shape : torch.Size([2, 10, 1000])

Fragmento representativo:

```

import math
import torch
import torch.nn as nn
class SinusoidalPositionalEncoding(nn.Module):
    def __init__(self, max_len, d_model):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log
    (10000.0) / d_model))

        pe[:, 0::2] = torch.sin(pos * div_term)
        pe[:, 1::2] = torch.cos(pos * div_term)

```

```
        self.register_buffer("pe", pe)

    def forward(self, x):
        return x + self.pe[:x.shape[1], :]


class LearnedPositionalEncoding(nn.Module):
    def __init__(self, max_len, d_model):
        super().__init__()
        self.pos_embedding = nn.Embedding(max_len, d_model)
        nn.init.normal_(self.pos_embedding.weight, mean=0.0, std=0.02)

    def forward(self, x):
        B, T, _ = x.shape
        positions = torch.arange(0, T, device=x.device)
```

10.2.4 Fuente práctica: 11_04_Submission_task.ipynb

Ruta: code/11_04_Submission_task/11_04_Submission_task.ipynb

Apartados detectados: 11_04 Submission task; Exercise 0 — Bring your building blocks; Exercise 1 — Verify ‘MultiHeadAttention‘ supports additive masks; Exercise 2 — Thread the encoder padding mask through ‘EncoderLayer‘ → ‘Encoder‘; Exercise 3 — Complete the ‘Decoder‘ masks.

Salidas gráficas almacenadas: 0.

Salida textual conservada:

> — Starting Training — | Epoch 1 | Loss: 2.9569 | Epoch 30 | Loss: 0.6204 | Epoch 60 | Loss: 0.0469

Fragmento representativo:

```

import math
import torch
import torch.nn as nn

class SinusoidalPositionalEncoding(nn.Module):
    def __init__(self, max_len, d_model):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log
    (10000.0) / d_model))

        pe[:, 0::2] = torch.sin(pos * div_term)
        pe[:, 1::2] = torch.cos(pos * div_term)

        self.register_buffer("pe", pe)

    def forward(self, x):
        return x + self.pe[:x.shape[1], :]

class LearnedPositionalEncoding(nn.Module):
    def __init__(self, max_len, d_model):
```

```
23          super().__init__()
24          self.pos_embedding = nn.Embedding(max_len, d_model)
25          nn.init.normal_(self.pos_embedding.weight, mean=0.0, std=0.02)
26
27      def forward(self, x):
28          B, T, _ = x.shape
29          positions = torch.arange(0, T, device=x.device)
```

10.3 Autoevaluación razonada

Pregunta

¿Por qué se escala el producto QK^T?

Solución. Para controlar su magnitud antes de softmax y evitar saturación cuando crece la dimensión de las claves.

Pregunta

¿Qué aporta la codificación posicional?

Solución. Añade información sobre el orden, que no estaría determinada por la atención sobre un conjunto de tokens sin posiciones.

10.4 Fuentes y trazabilidad

Diapositivas:

- `slides/11\_Attention\_mechanisms\_and\_transformers.pdf`

Nota personal: apuntes/Apuntes del Tema 11 - Attention Mechanisms and Transformers.md

11 Sistemas de recomendación

Este capítulo se construye a partir de las fuentes de diapositivas y notebook indicadas al final.

11.1 Conexión con las prácticas

Este tema no tiene una nota Markdown equivalente en el repositorio; se incorpora como capítulo documentado desde sus diapositivas y notebook, manteniendo la trazabilidad de la fuente.

11.1.1 Fuente práctica: `13_01_Recommender_systems.ipynb`

Ruta: code/13_01_Recommender_systems/13_01_Recommender_systems.ipynb

Apartados detectados: 13_01 Recommender systems; Exercise 1 — Dataset: MovieLens 100K, Download & Files; Exercise 2 — Train/Validation/Test Partition (Leave-Last-N per User); Exercise 3 — Content-Based Recommender; Exercise 4 — Ranking Metrics (Precision@K, Recall@K, NDCG@K).

Salidas gráficas almacenadas: 0.

Salida textual conservada:

> Users: 943, Items: 1682, Genres: 19 | Exercise 1: all checks passed | train (98114, 6) unique users 943 unique items 1678 | valid (943, 6) unique users 943 unique items 534

Fragmento representativo:

```

# !pip install torch pandas numpy scikit-learn requests

import copy
import math
import random
import zipfile
import requests
import numpy as np
import pandas as pd

from pathlib import Path
from typing import Callable, Dict, List, Tuple

from sklearn.preprocessing import normalize

import torch
import torch.nn as nn
```

```
from torch.utils.data import Dataset, DataLoader, RandomSampler

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

device = 'cuda' if torch.cuda.is_available() else 'cpu'

NUM_GENRES = 19
K = 10
```

11.2 Autoevaluación razonada

Pregunta

¿Qué diferencia hay entre filtrado colaborativo y basado en contenido?

Solución. El colaborativo usa patrones de interacción entre usuarios e ítems; el basado en contenido usa atributos de los ítems y preferencias observadas.

Pregunta

¿Por qué una métrica de recomendación debe definirse con cuidado?

Solución. Porque precisión, ranking, cobertura y diversidad miden objetivos diferentes y pueden entrar en conflicto.

11.3 Fuentes y trazabilidad

Diapositivas:

- `slides/13\_Recommender\_systems.pdf`